

# ML on Devices, Robots & Generative AI

## Day 5: Detailed learning notes

Reading heart signals, preparing models for small devices, learning through rewards, and generating text one token at a time.

---

### A companion to the slides and all four notebooks

ECG signals; TinyML and the ESP32; robots and reinforcement learning; generative AI from scratch.

### How to use these notes

Keep the relevant notebook open. Read the explanation, run its cell, and compare the output with the example. Follow the worked calculations with a calculator or a small Python expression. The exercises help you check that you understand what each setting changes. Prepared from the supplied

40-slide deck, speaker notes and four notebooks. Saved model outputs are identified as supplied results. The warehouse experiment and selected mathematical calculations were checked independently. The deep learning models were not retrained, and the ESP32 template was not tested on hardware for these notes.

## Contents

|           |                                                           |           |
|-----------|-----------------------------------------------------------|-----------|
| <b>1</b>  | <b>Starting the final day</b>                             | <b>4</b>  |
| 1.1       | How today's topics fit together . . . . .                 | 4         |
| 1.2       | The three-hour route . . . . .                            | 4         |
| 1.3       | What was checked for these notes . . . . .                | 4         |
| <b>2</b>  | <b>Body signals and the engineering measurement chain</b> | <b>5</b>  |
| 2.1       | What a signal represents . . . . .                        | 5         |
| 2.2       | From electrodes to an array . . . . .                     | 5         |
| 2.3       | Reading an ECG waveform . . . . .                         | 6         |
| <b>3</b>  | <b>Notebook 1: opening a real ECG recording</b>           | <b>7</b>  |
| 3.1       | Libraries and installation . . . . .                      | 7         |
| 3.2       | The three files and their contents . . . . .              | 7         |
| 3.3       | Sampling rate and the time axis . . . . .                 | 8         |
| 3.4       | Understanding the annotation symbols . . . . .            | 9         |
| <b>4</b>  | <b>Cleaning the signal with filters</b>                   | <b>9</b>  |
| 4.1       | Frequency, noise and the chosen passband . . . . .        | 9         |
| 4.2       | Adding a known 50 Hz disturbance . . . . .                | 10        |
| 4.3       | Applying the band-pass filter . . . . .                   | 11        |
| 4.4       | Exercise 1: remove slow drift . . . . .                   | 12        |
| <b>5</b>  | <b>Detecting beats and calculating heart rate</b>         | <b>13</b> |
| 5.1       | The peak-finding rules . . . . .                          | 13        |
| 5.2       | Matching detections to reference annotations . . . . .    | 13        |
| 5.3       | From sample gaps to beats per minute . . . . .            | 14        |
| 5.4       | What the printed HRV number actually measures . . . . .   | 15        |
| <b>6</b>  | <b>Classifying the shape of a heartbeat</b>               | <b>16</b> |
| 6.1       | A separate, prepared dataset . . . . .                    | 16        |
| 6.2       | Train and test splits; two baseline models . . . . .      | 17        |
| 6.3       | Why recall matters, and what else matters . . . . .       | 18        |
| <b>7</b>  | <b>The one-dimensional convolutional network</b>          | <b>18</b> |
| 7.1       | Architecture and tensor shapes . . . . .                  | 18        |
| 7.2       | Binary loss, training and inference . . . . .             | 19        |
| 7.3       | Working through the confusion matrix . . . . .            | 20        |
| <b>8</b>  | <b>TinyML: deciding where computation happens</b>         | <b>21</b> |
| 8.1       | Cloud, edge and a hybrid system . . . . .                 | 21        |
| 8.2       | Microcontrollers and small Linux computers . . . . .      | 21        |
| 8.3       | Flash, RAM and model size . . . . .                       | 21        |
| <b>9</b>  | <b>Notebook 2: training and converting the model</b>      | <b>22</b> |
| 9.1       | Setup and repeated data preparation . . . . .             | 22        |
| 9.2       | Creating a fixed-batch inference model . . . . .          | 24        |
| 9.3       | The float32 conversion . . . . .                          | 24        |
| 9.4       | The representative dataset . . . . .                      | 25        |
| 9.5       | Requesting full integer conversion . . . . .              | 25        |
| <b>10</b> | <b>Quantisation explained with numbers</b>                | <b>25</b> |

|           |                                                                  |           |
|-----------|------------------------------------------------------------------|-----------|
| 10.1      | Representing values on an integer grid . . . . .                 | 25        |
| 10.2      | Why the complete file does not shrink exactly fourfold . . . . . | 26        |
| <b>11</b> | <b>Running and inspecting the converted models</b>               | <b>27</b> |
| 11.1      | The interpreter loop . . . . .                                   | 27        |
| 11.2      | Interpreting the timing correctly . . . . .                      | 27        |
| 11.3      | Input metadata and converted operators . . . . .                 | 28        |
| 11.4      | Exercise: reducing the dense layer . . . . .                     | 29        |
| <b>12</b> | <b>Exporting the model and understanding the ESP32 template</b>  | <b>30</b> |
| 12.1      | Turning bytes into a C array . . . . .                           | 30        |
| 12.2      | Exporting input and output scales . . . . .                      | 30        |
| 12.3      | The supplied Arduino sketch . . . . .                            | 31        |
| 12.4      | What must be completed before using the sketch . . . . .         | 32        |
| 12.5      | The missing link between the sensor and the model . . . . .      | 32        |
| 12.6      | Raspberry Pi inference and file download . . . . .               | 33        |
| <b>13</b> | <b>Robotics and learning from rewards</b>                        | <b>33</b> |
| 13.1      | Sense, think and act . . . . .                                   | 33        |
| 13.2      | The reinforcement-learning vocabulary . . . . .                  | 34        |
| <b>14</b> | <b>Notebook 3: building the warehouse environment</b>            | <b>34</b> |
| 14.1      | Setup and state encoding . . . . .                               | 34        |
| 14.2      | Understanding the transition function . . . . .                  | 35        |
| 14.3      | The random baseline . . . . .                                    | 36        |
| <b>15</b> | <b>Q-learning, one update at a time</b>                          | <b>36</b> |
| 15.1      | What the Q-table stores . . . . .                                | 36        |
| 15.2      | A numerical update . . . . .                                     | 37        |
| 15.3      | Exploration and the training loop . . . . .                      | 37        |
| <b>16</b> | <b>Inspecting what the warehouse agent learned</b>               | <b>38</b> |
| 16.1      | The greedy route . . . . .                                       | 38        |
| 16.2      | The policy and the value map . . . . .                           | 39        |
| 16.3      | Exercise 1: changing reward design . . . . .                     | 41        |
| <b>17</b> | <b>The taxi environment in Gymnasium</b>                         | <b>42</b> |
| 17.1      | States, actions and rewards . . . . .                            | 42        |
| 17.2      | Evaluating a random driver . . . . .                             | 42        |
| 17.3      | Training for 5,000 episodes . . . . .                            | 43        |
| 17.4      | Watching a trip . . . . .                                        | 44        |
| 17.5      | Exercise 2: less training . . . . .                              | 45        |
| 17.6      | From tables to deep reinforcement learning . . . . .             | 45        |
| <b>18</b> | <b>Generative AI and next-token prediction</b>                   | <b>45</b> |
| 18.1      | Learning a distribution over possible outputs . . . . .          | 45        |
| 18.2      | What this small example teaches . . . . .                        | 46        |
| <b>19</b> | <b>Notebook 4: a counting language model</b>                     | <b>47</b> |
| 19.1      | Loading names and defining a vocabulary . . . . .                | 47        |
| 19.2      | Counting adjacent pairs . . . . .                                | 47        |
| 19.3      | From counts to conditional probabilities . . . . .               | 49        |
| 19.4      | Sampling a new name . . . . .                                    | 49        |
| 19.5      | Measuring next-token loss . . . . .                              | 50        |

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| <b>20 Building the neural name model</b>                        | <b>50</b> |
| 20.1 Creating contexts and targets . . . . .                    | 50        |
| 20.2 Splitting examples and defining the network . . . . .      | 51        |
| 20.3 Logits, cross-entropy and optimisation . . . . .           | 52        |
| 20.4 What this validation split can tell us . . . . .           | 52        |
| <b>21 Generating names and inspecting embeddings</b>            | <b>53</b> |
| 21.1 The generation function . . . . .                          | 53        |
| 21.2 What the embedding plot shows . . . . .                    | 53        |
| <b>22 Temperature and sampling behaviour</b>                    | <b>55</b> |
| 22.1 The mathematical effect . . . . .                          | 55        |
| 22.2 Reading the supplied experiment . . . . .                  | 55        |
| 22.3 Exercise 1: extreme settings . . . . .                     | 55        |
| 22.4 Exercise 2: extending the context . . . . .                | 56        |
| <b>23 From the name model to larger generative systems</b>      | <b>56</b> |
| 23.1 Attention and the training stages . . . . .                | 56        |
| 23.2 The optional Colab AI cell . . . . .                       | 57        |
| 23.3 Other tools mentioned in the slides . . . . .              | 57        |
| 23.4 Exercise 3: checking Nigerian names and meanings . . . . . | 58        |
| <b>24 Putting the five days together</b>                        | <b>58</b> |
| 24.1 What each day contributed . . . . .                        | 58        |
| 24.2 Choosing a project that can be finished . . . . .          | 58        |
| 24.3 Using generated assistance while learning . . . . .        | 59        |
| <b>25 Review questions and worked answers</b>                   | <b>59</b> |
| <b>26 Vocabulary reference</b>                                  | <b>60</b> |
| <b>27 Coverage, provenance and references</b>                   | <b>61</b> |
| 27.1 Reading the source material accurately . . . . .           | 61        |
| 27.2 Primary references . . . . .                               | 61        |

# 1 Starting the final day

*Material covered: Slides 1–3 and the speaker notes’ running order.*

## 1.1 How today’s topics fit together

On Day 4, we learned how a neural network transforms input values into a prediction. We examined weights, activations, loss, training and image features. Today we use those ideas in four settings.

First, we analyse a recording of the heart’s electrical activity and classify individual beat waveforms. Next, we convert that classifier into a compact format intended for embedded deployment. We then teach an agent to choose actions using rewards. Finally, we train a small model to generate names by predicting successive characters.

These examples involve different inputs and outputs, but each requires a clear task, an appropriate representation and an honest evaluation. A strong result on one classroom dataset answers a specific experimental question. The notes explain that question each time we report a number.

## 1.2 The three-hour route

| Time      | Topic         | Practical work                                |
|-----------|---------------|-----------------------------------------------|
| 0:00–0:10 | Recap         | Connect Day 4 to devices and sequences.       |
| 0:10–0:51 | Body signals  | Notebook 1: ECG filtering and classification. |
| 0:51–1:25 | TinyML        | Notebook 2: conversion and integer inference. |
| 1:25–1:35 | Break         | Questions and a pause.                        |
| 1:35–2:14 | Robots and RL | Notebook 3: warehouse and taxi agents.        |
| 2:14–2:52 | Generative AI | Notebook 4: counting and neural name models.  |
| 2:52–3:00 | Wrap-up       | Review and choose a next project.             |

The hands-on slides are 12, 20, 28 and 37. The detailed notes also support study after class, when there is more time to inspect each calculation. Run each notebook from the top in its own session. Variables such as `X`, `model`, `Q` and `b` are reused for different purposes across notebooks.

## 1.3 What was checked for these notes

The uploaded notebooks contain saved outputs. These provide the training accuracies, example predictions, conversion sizes and most figures below. TensorFlow training, model conversion, Colab’s AI service and hardware deployment were not rerun while preparing this document. We independently repeated the NumPy warehouse experiment and its reward exercises, and checked parameter counts, dimensions, probability calculations and quantisation arithmetic.

Some slide figures come from an earlier run. For example, Notebook 2 stores a 22.7 KiB integer model, while the slides report 23.3 KB. The relevant sections identify these differences. A saved result and an exercise expectation are also labelled separately.

## 2 Body signals and the engineering measurement chain

*Material covered: Slides 4–7 and 11; Notebook 1 introduction.*

### 2.1 What a signal represents

A signal records how a measured quantity changes. When we sample it repeatedly, we obtain a time series: an ordered sequence of numerical measurements. Each value needs context, including its unit, time, channel and measurement conditions.

A sensor does not directly provide a complete interpretation such as “this person is stressed”. It provides a measurement from which an interpretation may be estimated. Motion, poor contact, noise and differences between people can change the measurements.

| Signal           | What is measured and how it can be used                                                                                             |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| ECG              | Electrical potential differences associated with heart activity; used to study rhythm and waveform timing.                          |
| EEG              | Electrical activity measured through scalp electrodes; used in studies of sleep, brain activity and brain–computer interfaces.      |
| EMG              | Electrical activity associated with muscle activation; can support gesture recognition and prosthetic control.                      |
| PPG              | Changes in detected light associated with pulsatile blood-volume changes; used for pulse measurement.                               |
| SpO <sub>2</sub> | An estimate of oxygen saturation derived from suitable optical measurements and calibration, commonly using red and infrared light. |
| IMU              | Acceleration and angular velocity from an accelerometer and gyroscope; useful for motion and activity context.                      |
| Temperature      | A temperature measurement at a particular sensor location; interpretation depends on placement and conditions.                      |

PPG and ECG measure different physical quantities. Optical pulse peaks have a timing relationship with electrical heart activity, including the delay before the pulse reaches the measurement site. A temperature or motion feature may provide context for another signal.

The healthcare slide gives examples involving rhythm screening, medical images, malaria microscopy, sleep studies and prosthetic control. These require different kinds of input and labels. A waveform classifier, an image detector and a sleep-stage classifier answer different questions. A product’s clinical role also depends on the population, setting and validation evidence. The notebook’s binary beat classifier does not establish any of those wider capabilities.

### 2.2 From electrodes to an array

An ECG electrode arrangement measures small voltage differences at the skin. An analogue front end amplifies and conditions these voltages. Analogue filtering helps control unwanted components before conversion. An analogue-to-digital converter, or ADC, maps the input voltage to a numerical code. Software then performs further processing and, if needed, model inference.

The slides use a roughly 1 mV signal and a gain around 1,000 as an illustration. With that gain, a 1 mV change would produce approximately a 1 V change before accounting for offsets, filter response and saturation. The AD8232’s instrumentation amplifier has a gain of 100; the surrounding circuit can add gain. Its overall gain must be obtained from the actual board design, as the manufacturer’s examples show [3].

A  $B$ -bit ADC provides  $2^B$  possible codes. For an ideal 12-bit conversion spanning 3.3 V, one step is approximately  $3.3/4096 = 0.000806$  V. Actual ESP32 input ranges and conversion accuracy depend on attenuation, calibration and the device. An ADC count is not automatically a millivolt value at the electrodes.

The notebook begins with an already-recorded public dataset, so students do not need to connect electrodes to themselves. Any body-connected hardware activity needs an appropriately isolated, supervised setup. Battery power alone does not ensure isolation if a USB cable or other equipment creates another electrical connection. This classroom classifier is for learning and has not been validated for clinical decisions.

### 2.3 Reading an ECG waveform

The heart has two atria and two ventricles. The P wave primarily represents atrial depolarisation, the electrical activation associated with atrial contraction. The QRS complex represents ventricular depolarisation. The T wave primarily represents ventricular repolarisation, the electrical recovery process. Mechanical contraction follows the electrical activity; the plotted ECG records electrical potential differences.

Q, R and S name deflections within the QRS complex. The tall positive R wave is convenient for timing in this particular recording. Its prominence and polarity vary with the lead and beat morphology. A peak-finding rule designed for a clear upright R wave needs revision for other recordings.

### 3 Notebook 1: opening a real ECG recording

#### 3.1 Libraries and installation

```
!pip install -q wfdb
```

`wfdb` reads waveform and annotation files used by PhysioNet. The exclamation mark executes an installation command in the notebook's shell, and `-q` reduces its output. Installation requires access to the package source.

```
import numpy as np
import matplotlib.pyplot as plt
import urllib.request
import wfdb
from scipy.signal import butter, filtfilt, iirnotch, find_peaks
plt.rcParams["figure.figsize"] = (14, 4)
plt.rcParams["font.size"] = 12
print("Ready!")
```

NumPy supplies numerical arrays, Matplotlib draws figures, and `urllib.request` downloads files. SciPy provides the filter-design, filtering and peak-finding functions. `rcParams` sets default figure size and text size. These plotting settings change presentation, while the signal values remain the same.

#### 3.2 The three files and their contents

```
base = "https://raw.githubusercontent.com/MIT-LCP/wfdb-python/main/sample-data/"
for ext in ["hea", "dat", "atr"]:
    urllib.request.urlretrieve(base + "100." + ext, "100." + ext)

record = wfdb.rdrecord("100")          # the signal
ann = wfdb.rdann("100", "atr")         # the doctors' labels

fs = record.fs                        # sampling rate
ecg = record.p_signal[:, 0]           # channel 0 = lead MLII, in millivolts
print("Sampling rate:", fs, "samples per second")
print("Channels:", record.sig_name)
print("Length:", len(ecg), "samples =", round(len(ecg) / fs / 60, 1), "minutes")
```

The loop downloads record 100 with three extensions. The `.hea` file describes the recording, including channel and sampling information. The `.dat` file stores signal samples in the specified encoding. The `.atr` file stores annotations. Keeping them together allows WFDB to interpret the data.

`rdrecord("100")` returns the signal record. `rdann("100", "atr")` reads its annotations. The recording contains two channels, MLII and V5. The slice `record.p_signal[:, 0]` selects all rows of the first channel. `p_signal` supplies physical signal values, here millivolts, using the recording's calibration information.

The saved output reports 650,000 samples per channel and a sampling rate of 360 samples per second. The corresponding duration is

$$\frac{650000}{360} = 1805.56 \text{ s}, \quad \frac{650000}{360 \times 60} = 30.09 \text{ min.}$$

This is one record from MIT-BIH, a database of two-channel ambulatory recordings with expert-reviewed annotations [1]. The classifier later in the notebook uses a different dataset.

### 3.3 Sampling rate and the time axis

```
t = np.arange(len(ecg)) / fs          # time in seconds
plt.plot(t[:10 * fs], ecg[:10 * fs])
plt.xlabel("time (s)"); plt.ylabel("mV"); plt.title("First 10 seconds of record 100")
plt.show()
```

`np.arange(len(ecg))` creates the sample indices  $0, 1, 2, \dots$ . Dividing by `fs` converts each index to seconds. At 360 samples per second, adjacent samples are separated by

$$\Delta t = \frac{1}{360} \text{ s} \approx 2.78 \text{ ms.}$$

The first ten seconds contain  $10 \times 360 = 3600$  samples. The slice stops before index 3600, so the final displayed sample is just before 10 seconds. The waveform's vertical axis is millivolts.

Some readers may obtain a floating-point `record.fs`. Python slice bounds must be integers. A robust form is `n10=int(10*fs)`, followed by `t[:n10]` and `ecg[:n10]`. The sampling rate itself can remain a floating-point number for calculations.

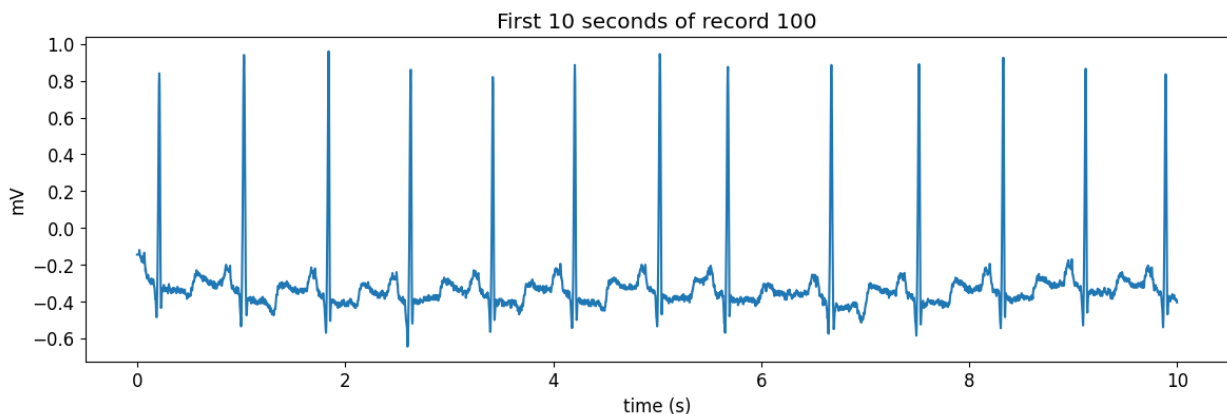


Figure 1: The first ten seconds saved in Notebook 1. Each prominent QRS complex provides a candidate timing landmark.

```
# Zoom in on one beat
plt.figure(figsize=(8, 4))
seg = slice(int(0.5 * fs), int(1.4 * fs))
plt.plot(t[seg], ecg[seg], linewidth=2)
plt.xlabel("time (s)"); plt.ylabel("mV"); plt.title("One heartbeat: P, QRS, T")
plt.show()
```

`slice(int(0.5*fs), int(1.4*fs))` describes a 0.9-second window starting at 0.5 seconds. Using the same slice for time and voltage keeps the plotted pairs aligned. The zoom helps you see the smaller features around the prominent QRS complex.

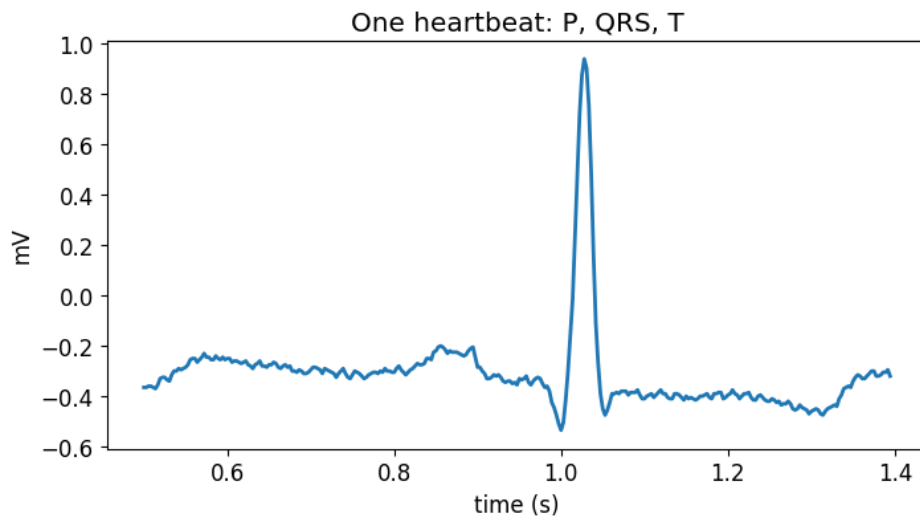


Figure 2: The supplied zoomed waveform. The title names the principal ECG components; their exact boundaries are not annotated in this plot.

### 3.4 Understanding the annotation symbols

```
# What did the cardiologists label? N = normal, A = atrial premature beat, V = ventricular
  premature beat
import collections
print(collections.Counter(ann.symbol))
```

`Counter` totals repeated symbols. This record contains 2,239 N annotations, 33 A annotations and one V annotation: normal, atrial premature and ventricular premature beat labels. There is also one + rhythm marker. A rhythm marker is an annotation event and should not be counted as an extra heartbeat.

The beat count is  $2239 + 33 + 1 = 2273$ . The annotation sample indices tell us where events occur in the recording. The classifier's later binary labels are not created by extracting these 2,273 annotations.

## 4 Cleaning the signal with filters

*Material covered: Slide 8; Notebook 1, Part 2.*

### 4.1 Frequency, noise and the chosen passband

Frequency describes repetition per second. A sinusoid at 50 Hz completes 50 cycles in one second. Slow baseline drift changes over much longer intervals; a 0.2 Hz drift completes one cycle every five seconds. Electrode motion, mains coupling and muscle activity can introduce unwanted components, with overlapping frequency content.

A band-pass filter attenuates components below and above a chosen range. The notebook's 0.5–40 Hz band is a practical demonstration for this task. ECG information and noise do not occupy perfectly separate frequency bands, so filtering can alter useful morphology as well as interference. Appropriate settings depend on the intended analysis.

The Nyquist frequency here is  $360/2 = 180$  Hz. The requested digital filter frequencies are below it. Analogue filtering before sampling remains relevant because digital filtering cannot undo arbitrary aliasing already introduced during acquisition.

## 4.2 Adding a known 50 Hz disturbance

```
noisy = ecg + 0.3 * np.sin(2 * np.pi * 50 * t)      # add mains hum

b, a = iirnotch(w0=50, Q=30, fs=fs)                # notch filter: removes exactly 50 Hz
cleaned = filtfilt(b, a, noisy)

fig, axes = plt.subplots(3, 1, figsize=(14, 8), sharex=True)
seg = slice(0, 3 * fs)
axes[0].plot(t[seg], ecg[seg]); axes[0].set_title("Original")
axes[1].plot(t[seg], noisy[seg], color="tab:red"); axes[1].set_title("With 50 Hz mains noise")
axes[2].plot(t[seg], cleaned[seg], color="tab:green"); axes[2].set_title("After the 50 Hz notch
    filter")
axes[2].set_xlabel("time (s)"); plt.tight_layout(); plt.show()
print("Average error after cleaning:", round(np.abs(cleaned - ecg)[1000:-1000].mean(), 4), "mV")
```

The expression  $0.3 \sin(2\pi 50t)$  generates a sinusoidal disturbance with amplitude 0.3 mV. Multiplication by  $2\pi$  converts cycles to radians for the sine function. Adding this to the original recording makes an experiment with a known reference signal.

`iirnotch` designs a narrow rejection band centred at 50 Hz. The quality factor  $Q = 30$  corresponds to a design bandwidth of approximately  $50/30 = 1.67$  Hz [4]. A notch affects a neighbourhood around its centre. The notebook’s wording “removes exactly 50 Hz” is a simplified description of its target frequency.

The arrays `b` and `a` contain the filter’s numerator and denominator coefficients. `filtfilt` applies forward and backward filtering, producing zero net phase shift for the combined filter and squaring its magnitude response [5]. This uses a stored segment, including future samples relative to each point. A streaming implementation needs a causal filter and must account for its timing behaviour.

The figure uses three vertically stacked axes with a shared time axis. Red shows the deliberately contaminated signal, and green shows the filtered result. The first few samples can contain a transient while the filter handles the record boundary.

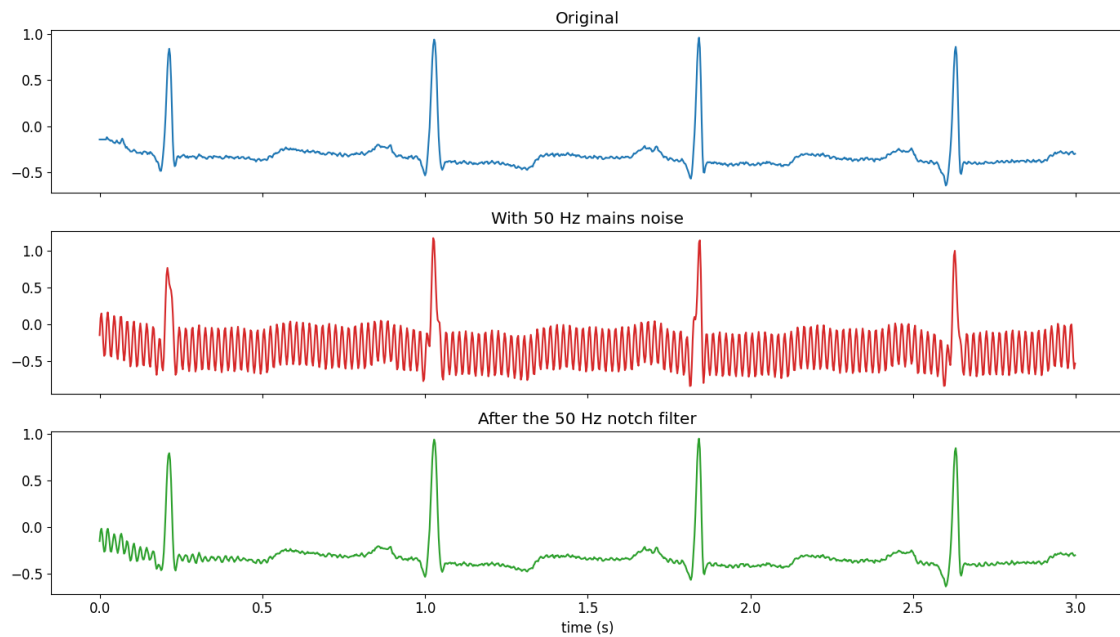


Figure 3: Original signal, added interference and notch-filtered signal from the supplied run.

The mean absolute difference is computed after excluding 1,000 samples at each end. This removes about 2.78 seconds from each boundary. The stored result is 0.0036 mV. It measures reconstruction error relative to this particular original recording, which can itself contain noise. It is not a general guarantee about ECG denoising.

### 4.3 Applying the band-pass filter

```
b, a = butter(3, [0.5, 40], btype="band", fs=fs) # band-pass 0.5-40 Hz
ecg_f = filtfilt(b, a, ecg)

seg = slice(0, 10 * fs)
plt.plot(t[seg], ecg[seg], alpha=0.6, label="raw")
plt.plot(t[seg], ecg_f[seg], label="band-pass 0.5-40 Hz")
plt.legend(); plt.xlabel("time (s)"); plt.title("Band-pass filter removes the slow drift"); plt.
show()
```

`butter` designs a Butterworth filter with a smooth passband. `[0.5,40]` supplies its two cutoff frequencies in hertz because `fs` is supplied. For a band-pass design, the prototype order 3 produces an order-6 filter before the forward/backward application [6]. The two-pass operation further changes the combined response. Describing this simply as a third-order streaming filter would hide these details.

This cell filters `ecg`, the original signal. It does not apply the band-pass to the preceding `cleaned` array. The two cells demonstrate related filtering operations through separate branches.

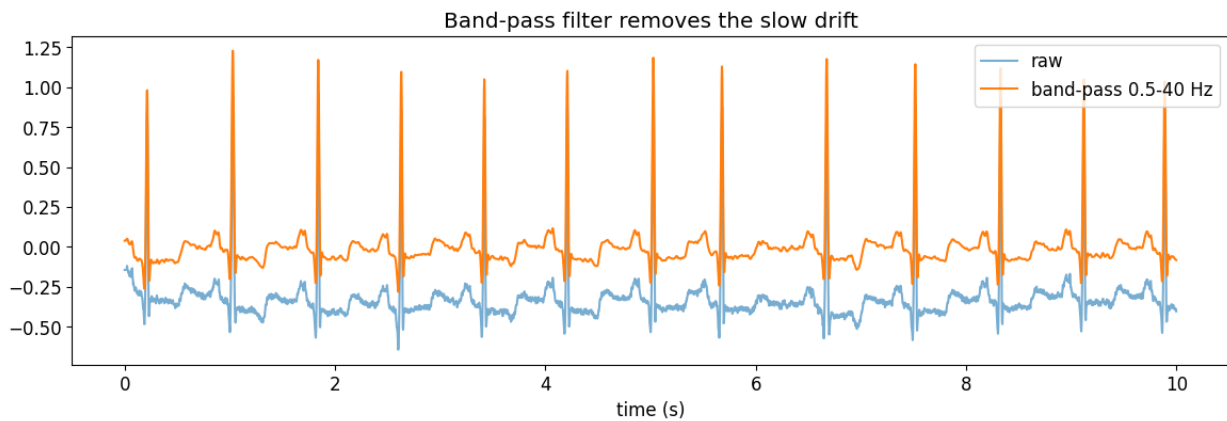


Figure 4: The saved raw and band-pass-filtered traces. The shift in the baseline is visible.

For a numerically robust general-purpose implementation, SciPy supports second-order sections through `output="sos"`, with `sosfiltfilt` for offline filtering. This changes the numerical representation of the filter. It does not remove the need to choose appropriate passband and boundary handling.

#### 4.4 Exercise 1: remove slow drift

```
# Exercise 1
drift = 0.5 * np.sin(2 * np.pi * 0.2 * t)
wandering = ecg + drift
fixed = filtfilt(b, a, wandering)      # b, a = the band-pass filter from Part 2
seg = slice(0, 10 * fs)
plt.plot(t[seg], wandering[seg], label="with breathing drift")
plt.plot(t[seg], fixed[seg], label="after band-pass")
plt.legend(); plt.show()
```

The added drift has amplitude 0.5 mV and frequency 0.2 Hz, below the lower cutoff. The band-pass should strongly reduce it. Compare the result with the original filtered ECG, allowing for boundary effects and the filter's non-ideal response.

The solution depends on `b,a` still referring to the band-pass design. If you rerun the notch-design cell immediately before it, these names will instead hold notch coefficients. Names such as `b_band,a_band` and `b_notch,a_notch` make this dependency easier to manage. The uploaded solution cell has no saved output.

## 5 Detecting beats and calculating heart rate

### 5.1 The peak-finding rules

```
peaks, _ = find_peaks(ecg_f, height=0.5, distance=int(0.25 * fs))
print("Heartbeats found:", len(peaks))

seg_peaks = peaks[peaks < 10 * fs]
plt.plot(t[:10 * fs], ecg_f[:10 * fs])
plt.plot(t[seg_peaks], ecg_f[seg_peaks], "rv", markersize=12)
plt.title(f"{len(seg_peaks)} R peaks in the first 10 seconds"); plt.xlabel("time (s)"); plt.show()
```

`find_peaks` selects local maxima. `height=0.5` requires a filtered amplitude of at least 0.5 mV. `distance=int(0.25*fs)` requires accepted peaks to be at least 90 samples apart. The first return value contains sample indices, and the second contains associated properties. The underscore means the code does not use that second value.

The 0.25-second spacing corresponds to 240 beats per minute. This is a detector setting, not a physiological claim that faster rhythms are impossible. It can suppress true closely spaced beats. The fixed height also depends on lead, polarity, gain and noise.

The saved detector finds 2,273 peaks in the complete recording, including 13 in the first ten seconds. The Boolean selection `peaks < 10*fs` keeps early peaks for plotting. "rv" requests red downward triangle markers.

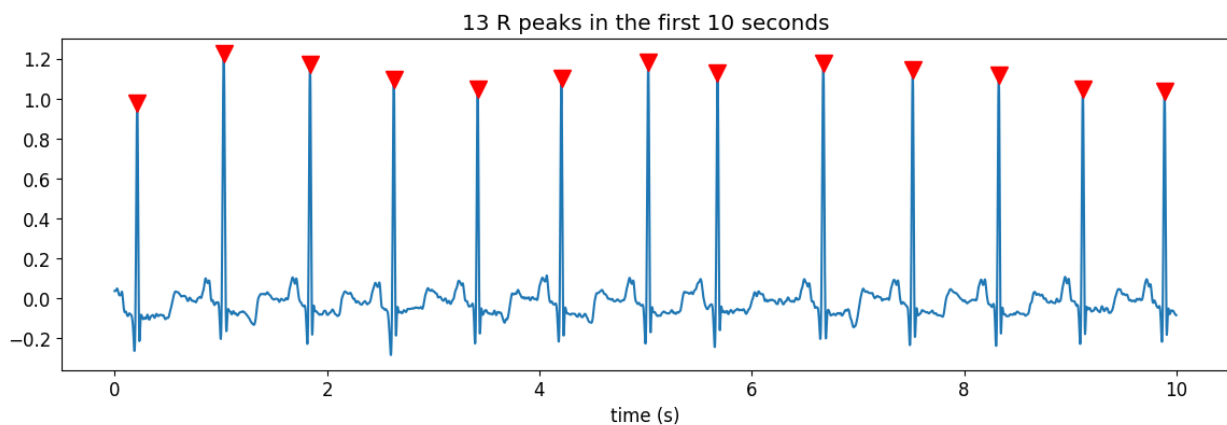


Figure 5: Detected peaks in the first ten seconds, as stored in Notebook 1.

### 5.2 Matching detections to reference annotations

```
beat_symbols = set("NLRaAJSVFejE/fQ")
true_beats = np.array([s for s, sym in zip(ann.sample, ann.symbol) if sym in beat_symbols])

matched = sum(np.min(np.abs(true_beats - p)) < 0.05 * fs for p in peaks)
print("Real beats (doctors):", len(true_beats))
print("Correctly found:      ", matched)
print("Missed:               ", len(true_beats) - matched)
print("False alarms:         ", len(peaks) - matched)
```

`beat_symbols` lists annotation symbols treated as beats. Pairing sample indices and symbols with

`zip` lets the comprehension retain only beat positions. For every detected peak  $p$ , the code finds the smallest absolute distance to any reference beat and checks whether that distance is strictly less than  $0.05f_s = 18$  samples.

The recorded summary is 2,272 matches, one missed beat and one false alarm. For a general evaluation, matching should be one-to-one: each detected event and each reference event can be used at most once. The displayed implementation counts detections near any reference without explicitly enforcing this. Counts alone do not verify that association rule across arbitrary datasets.

With one-to-one matches, precision would be  $TP/(TP + FP)$  and recall would be  $TP/(TP + FN)$ . Substituting the reported counts gives approximately  $2272/2273 = 99.96\%$  for both. This result applies to this recording and its tolerance rule. It does not demonstrate reliable detection on every ECG lead or patient.

### 5.3 From sample gaps to beats per minute

```
rr = np.diff(peaks) / fs          # time between beats (seconds)
hr = 60 / rr                     # beats per minute

print(f"Average heart rate: {hr.mean():.1f} bpm    (min {hr.min():.0f}, max {hr.max():.0f})")
print(f"Heart rate variability (SDNN): {rr.std() * 1000:.0f} ms")

minutes = peaks[1:] / fs / 60
plt.plot(minutes, hr, ".", markersize=3)
plt.xlabel("time (minutes)"); plt.ylabel("bpm"); plt.title("Heart rate, beat by beat, over 30
minutes")
plt.show()
```

`np.diff(peaks)` subtracts neighbouring peak indices. Dividing by the sampling rate gives RR intervals in seconds. The name RR refers to the interval between successive R-wave landmarks. For a gap of 288 samples,

$$RR = 288/360 = 0.8 \text{ s}, \quad HR = 60/0.8 = 75 \text{ bpm}.$$

A shorter interval yields a larger rate. With 2,273 peaks, there are 2,272 consecutive intervals. `peaks[1:]` associates each interval-derived rate with the later of its two peaks. Dividing that time by 60 places the plot's horizontal axis in minutes.

The stored mean of the interval-derived rates is 75.8 bpm, with minimum and maximum values rounded to 58 and 115 bpm. The mean of  $60/RR$  generally differs from  $60/\text{mean}(RR)$  because reciprocal transformation is nonlinear. For intervals of 0.5 and 1 second, the first calculation gives  $(120 + 60)/2 = 90$  bpm, while the second gives  $60/0.75 = 80$  bpm.

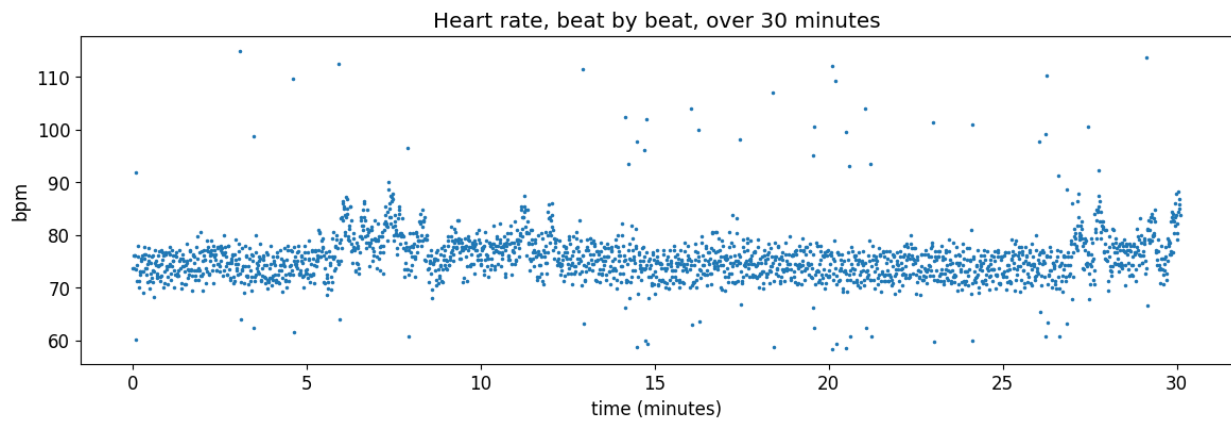


Figure 6: The stored interval-derived rate over the recording. A short RR interval raises the plotted value.

Premature beats can produce short intervals, and the annotations identify premature beats in this record. An isolated high point should be checked against its annotation and waveform before assigning a cause. Detection errors can also shorten or lengthen measured intervals.

#### 5.4 What the printed HRV number actually measures

The expression `rr.std()*1000` computes the population standard deviation of all detected RR intervals in milliseconds. NumPy's default divides by the number of intervals. The stored result is about 48 ms.

SDNN refers to the standard deviation of normal-to-normal intervals. The cell does not remove intervals affected by premature beats or detection errors before calculating its value. A precise label for this output is therefore “standard deviation of detected RR intervals”. Computing a defensible SDNN estimate requires appropriate normal-beat selection, artefact handling and a specified interval-estimation convention [7].

If an abnormal beat lies between two normal beats, simply deleting its peak and joining the normal peaks creates a long interval across the intervening beat. That is not a consecutive normal-to-normal interval. Interval-level validity needs to preserve the original sequence and identify suitable adjacent beats.

## 6 Classifying the shape of a heartbeat

### 6.1 A separate, prepared dataset

```
import pandas as pd

# ECG5000: 4,998 heartbeats, 140 samples each. Last column: 1 = normal, 0 = abnormal.
# Same file as TensorFlow's official tutorial; if Google's link is blocked we use a GitHub copy.
urls = ["http://storage.googleapis.com/download.tensorflow.org/data/ecg.csv",
        "https://raw.githubusercontent.com/kanesp/ECG_Anomaly-Detection/master/data/ecg.csv"]
for url in urls:
    try:
        df = pd.read_csv(url, header=None)
        print("Loaded from", url)
        break
    except Exception as e:
        print("Could not load", url, "-- trying the next link")

X = df.iloc[:, :140].values.astype("float32")
y = (df.iloc[:, 140] == 0).astype(int).values      # we flip it: 1 = ABNORMAL (the thing we want
                                                    # to catch)
print("Beats:", X.shape, "  abnormal:", y.sum(), "  normal:", (y == 0).sum())
```

The CSV used here contains 4,998 prepared waveforms with 140 numerical entries each and one label column. The first download attempt uses the TensorFlow tutorial's data file; the second is a GitHub copy. `header=None` tells pandas that the first row is data. The loop stops after a successful load.

If every download fails, there is no newly loaded table. Add an explicit failure check before continuing in a reusable script; an old `df` remaining in an interactive session can otherwise hide a failed download. The saved notebook loaded the first URL successfully, while the speaker notes describe using the fallback.

The original ECG5000 archive has 5,000 examples, five classes and a supplied 500/4,500 train/test split. Its source is a single long recording, and its beats were extracted and interpolated to equal length [8]. Today's 4,998-row binary CSV is a tutorial variant, with a newly constructed split. Keep these dataset descriptions distinct.

`iloc[:, :140]` selects columns zero through 139. `astype("float32")` changes the storage type. The final column in this CSV marks normal as 1 and abnormal as 0. The expression `df.iloc[:, 140] == 0` is true for abnormal beats; converting that Boolean value to an integer creates the new convention 1 = abnormal and 0 = normal.

The saved class counts are 2,079 abnormal and 2,919 normal. These labels describe the provided beat categories. A normal-labelled beat can come from a patient who has a broader cardiac condition.

The 140 values are positions within a prepared waveform. They are not automatically 140 consecutive raw samples at 360 Hz. Do not carry the earlier record's sampling rate into this dataset without evidence about its preparation. This distinction becomes essential when connecting a live sensor to the model.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 4), sharey=True)
for ax, label, name in zip(axes, [0, 1], ["Normal", "Abnormal"]):
    for beat in X[y == label][:15]:
        ax.plot(beat, alpha=0.5)
    ax.set_title(f"{name} heartbeats (15 examples)")
plt.show()
```

`X[y==label]` keeps waveforms from one class, and `[:15]` selects its first 15 examples. Each row is plotted as a line. Transparency helps reveal overlap. The horizontal axis is waveform position, and the vertical values follow the prepared dataset's scaling.

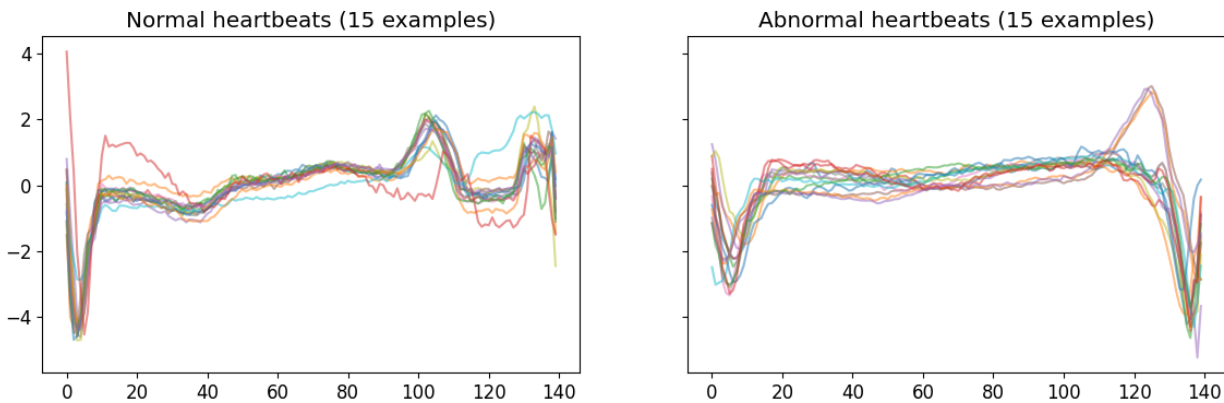


Figure 7: Prepared normal and abnormal examples saved in the notebook. These are illustrative selections from the data.

## 6.2 Train and test splits; two baseline models

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, recall_score, confusion_matrix

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
                                                    stratify=y)
print("Train:", len(X_train), " Test:", len(X_test))

results = {}
for name, model in [("Logistic regression", LogisticRegression(max_iter=2000)),
                    ("Random forest", RandomForestClassifier(n_estimators=200, random_state=42))
]:
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    results[name] = model
    print(f"{name:20s} accuracy {accuracy_score(y_test, pred):.1%}    abnormal beats caught (
        recall) {recall_score(y_test, pred):.1%}")
```

`test_size=0.2` requests a fifth of the rows for testing, producing 3,998 training and 1,000 test examples here. `stratify=y` approximately preserves class proportions. The fixed random seed makes the selection repeatable under the same implementation and data.

Logistic regression learns a weighted combination of the 140 inputs and maps its score into a binary probability. Its name contains “regression”, but here it performs classification. `max_iter=2000` increases the optimisation iteration limit. A random forest combines predictions from 200 decision

trees; its splits can represent nonlinear relationships among waveform positions.

The `results` dictionary stores fitted model objects using readable names as keys. `fit` adjusts parameters from training data, `predict` returns class labels, and the metric functions compare them with held-out labels. The saved outputs are 99.3% accuracy and 98.6% abnormal recall for logistic regression, and 99.5% accuracy and 98.8% abnormal recall for the forest.

Because these waveforms originate from one patient recording, a random beat split evaluates held-out beats within that source. It cannot establish performance on unseen patients. Evaluating a device requires representative data and an appropriate patient-level split. A larger network alone does not solve that evaluation problem.

### 6.3 Why recall matters, and what else matters

Abnormal recall is  $TP/(TP + FN)$ : the fraction of abnormal examples successfully detected. Missing relevant events can have serious consequences. False alarms also matter because they can overwhelm users or clinical workflows. The balance depends on the intended use and should be evaluated through recall, precision, false-alarm burden and other task-specific evidence.

Accuracy measures all correct labels together. Here the positive class is explicitly abnormal, which makes the default binary recall calculation match the intended quantity. Always establish the label convention before interpreting a score.

## 7 The one-dimensional convolutional network

### 7.1 Architecture and tensor shapes

```
import keras
from keras import layers
keras.utils.set_random_seed(42)

cnn = keras.Sequential([
    keras.Input(shape=(140, 1)),
    layers.Conv1D(16, 7, activation="relu"),
    layers.MaxPooling1D(4),
    layers.Conv1D(32, 5, activation="relu"),
    layers.MaxPooling1D(2),
    layers.Flatten(),
    layers.Dense(32, activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(1, activation="sigmoid"),      # probability of ABNORMAL
])
cnn.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
print("Weights:", cnn.count_params())
history = cnn.fit(X_train[..., None], y_train, epochs=20, batch_size=32, validation_split=0.1,
                  verbose=0)

pred = (cnn.predict(X_test[..., None], verbose=0)[: , 0] > 0.5).astype(int)
print(f"1-D CNN                accuracy {accuracy_score(y_test, pred):.1%}    recall {recall_score(
    y_test, pred):.1%}")
print("Confusion matrix [[TN FP] [FN TP]]:\n", confusion_matrix(y_test, pred))
```

A one-dimensional convolution slides along waveform position. Each filter learns a short pattern across neighbouring values. This is closely related to the image convolution from Day 4, with one spatial or temporal axis. Learned filters may help distinguish waveform shapes, but their clinical

meaning needs evidence.

`x_train[..., None]` adds a channel dimension. The input changes from (3998, 140) to (3998, 140, 1): examples, positions, channels. The network expects 140 positions and one channel per example.

| Layer                        | Output per example | Parameters |
|------------------------------|--------------------|------------|
| Input                        | $140 \times 1$     | 0          |
| Conv1D: 16 filters, length 7 | $134 \times 16$    | 128        |
| MaxPool: size 4              | $33 \times 16$     | 0          |
| Conv1D: 32 filters, length 5 | $29 \times 32$     | 2,592      |
| MaxPool: size 2              | $14 \times 32$     | 0          |
| Flatten                      | 448                | 0          |
| Dense ReLU                   | 32                 | 14,368     |
| Dropout                      | 32                 | 0          |
| Dense sigmoid                | 1                  | 33         |
| Total                        |                    | 17,121     |

For a valid convolution with stride one, output length is input length minus kernel length plus one. Thus  $140 - 7 + 1 = 134$ , followed by pooling to  $\lfloor 134/4 \rfloor = 33$ . The second convolution gives  $33 - 5 + 1 = 29$ , followed by pooling to 14. Pooling drops unmatched edge positions under these settings.

A convolution has  $(KC_{\text{in}} + 1)C_{\text{out}}$  parameters including biases. The first has  $(7 \times 1 + 1)16 = 128$  and the second  $(5 \times 16 + 1)32 = 2592$ . Flattening gives  $14 \times 32 = 448$  values. The hidden dense layer has  $(448 + 1)32 = 14368$  parameters, and the output has  $(32 + 1)1 = 33$ .

Most parameters are in the dense layer. This observation motivates the smaller-layer exercise in Notebook 2. Dropout adds no learned parameters; it randomly suppresses selected activations during training with rate 0.3.

## 7.2 Binary loss, training and inference

The single sigmoid output estimates the model's probability for abnormal. Binary cross-entropy for target  $y \in \{0, 1\}$  and prediction  $p$  is

$$L = -[y \log p + (1 - y) \log(1 - p)].$$

For an abnormal beat,  $y = 1$ , so the loss is  $-\log p$ . Assigning  $p = 0.9$  produces approximately 0.105, while  $p = 0.1$  produces approximately 2.303. The latter prediction gives the true class much less probability.

Adam updates the parameters using gradients. Training runs for 20 epochs with batches of 32 and a validation fraction of 0.1. Keras reserves the last portion of the provided training arrays for validation before shuffling the fitting portion. Here that leaves 3,598 fitting examples and 400 validation examples. A fitting epoch has  $\lceil 3598/32 \rceil = 113$  updates.

`verbose=0` suppresses progress logs. `predict` returns one output value per example, and `[:, 0]` removes its final length-one dimension. The strict test `>0.5` selects abnormal; a value exactly equal to 0.5 is assigned normal by this code.

### 7.3 Working through the confusion matrix

The saved CNN output is

$$\begin{pmatrix} 582 & 2 \\ 5 & 411 \end{pmatrix} = \begin{pmatrix} TN & FP \\ FN & TP \end{pmatrix}.$$

Rows are true labels and columns predicted labels, ordered normal then abnormal. There are 584 normal and 416 abnormal test examples. The calculations are

$$\text{accuracy} = \frac{582 + 411}{1000} = 99.3\%, \quad \text{recall} = \frac{411}{416} \approx 98.8\%,$$

$$\text{precision} = \frac{411}{411 + 2} \approx 99.5\%.$$

Five abnormal examples were missed and two normal examples generated false alarms. Equal rounded accuracy across models can hide different mistakes. A classification probability also needs calibration evidence before being interpreted as a reliable risk estimate in a new population.

```
# Look at the beats the CNN got wrong
wrong = np.where(pred != y_test)[0]
plt.figure(figsize=(14, 4))
for i in wrong:
    plt.plot(X_test[i], label=f"true: {'abnormal' if y_test[i] else 'normal'}")
plt.title(f"The {len(wrong)} beats the CNN got wrong"); plt.legend(fontsize=9); plt.show()
```

`np.where(pred != y_test)[0]` returns the indices of disagreements. The loop plots those waveforms and labels their true class. The saved plot contains seven examples. Inspecting these can suggest questions about noise, labels or unusual shapes; conclusions should be tested without repeatedly tailoring the model to the final test set.

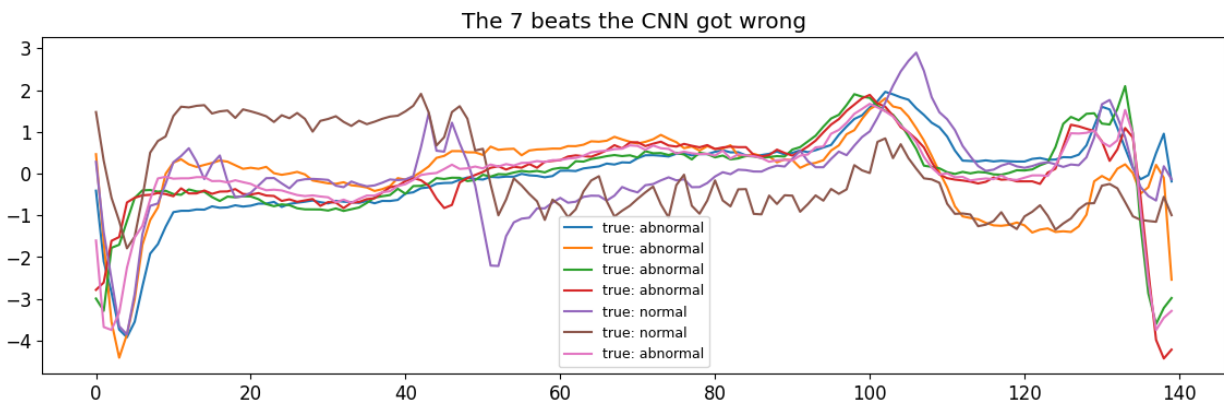


Figure 8: The seven test errors saved for the CNN.

## 8 TinyML: deciding where computation happens

*Material covered: Slides 13–16 and 19; Notebook 2 introduction.*

### 8.1 Cloud, edge and a hybrid system

Cloud inference sends information to a remote computer that runs the model and returns an answer. Edge inference performs computation near the source, for example on a watch, phone, microcontroller or nearby computer. A hybrid design divides the work: a wearable may extract simple features locally and send summaries to a phone for display or longer-term analysis.

The warm-up question about a smartwatch therefore has several possible answers. The location depends on the function and product design. Counting a pulse or step can also use conventional signal processing, so local processing does not necessarily involve a learned model.

Local inference can operate without a network connection once its required model and data are available. It can reduce transmission and provide predictable response times. Privacy still depends on what is stored, transmitted and accessible elsewhere in the system. Latency and battery life need measurement. The slide’s phrases “instant” and “months on a battery” describe design goals whose achievement depends on the complete implementation.

### 8.2 Microcontrollers and small Linux computers

A microcontroller integrates a processor, memory and hardware interfaces for embedded tasks. It can run bare-metal firmware or a real-time operating system. ESP32 development commonly uses an RTOS, so “no operating system” is too broad a description of microcontrollers.

The classic ESP32 family in the lesson has up to 240 MHz processing and 520 KB on-chip SRAM [9]. The example development board has a 4 MB flash configuration. Other ESP32 variants and boards have different resources. The operating system, communication stacks, buffers and application also consume memory.

A Raspberry Pi is a single-board computer that can run Linux, Python and multiple applications. A Jetson platform adds GPU-oriented computing for edge workloads. The Arduino Nano 33 BLE Sense is another microcontroller-based teaching board with onboard sensors. The slide’s memory figures refer to selected configurations, so choose hardware using the exact model’s specifications.

The approximate 30,000-times RAM comparison assumes a laptop around 16 GB and an ESP32 around 520 KB. Its purpose is to show the scale difference. Available application memory and required execution resources are more useful deployment quantities than a marketing memory total.

### 8.3 Flash, RAM and model size

Flash stores persistent firmware and model bytes. RAM holds changing values while a program runs, including intermediate tensors, input buffers, stacks and runtime state. A 23 KB model file can require additional RAM and compiled code space.

The TensorFlow Lite Micro interpreter uses a tensor arena: a preallocated working-memory region for tensors and temporary allocations. The notebook chooses 20 KiB for this arena without showing a board-level measurement that it is sufficient. Allocation success and complete application memory use must be checked on the target.

An embedded C array packages model bytes with firmware. Some microcontrollers also support flash file systems or removable storage.

## 9 Notebook 2: training and converting the model

### 9.1 Setup and repeated data preparation

```
import numpy as np
import matplotlib.pyplot as plt
import os, time, warnings
import tensorflow as tf
import keras
from keras import layers
warnings.filterwarnings("ignore")
plt.rcParams["font.size"] = 12
print("TensorFlow", tf.__version__)
```

TensorFlow supplies the converter and interpreter APIs used by the notebook. Keras defines and trains the network. `os` reads file sizes and directory entries; `time` measures elapsed duration. The saved environment reports TensorFlow 2.20.0.

`warnings.filterwarnings("ignore")` suppresses warning messages. While this keeps the demonstration output shorter, warnings can contain useful compatibility information. During debugging, allow them to appear and read their content before deciding which are harmless.

```
import pandas as pd

# ECG5000: 4,998 heartbeats, 140 samples each. Last column: 1 = normal, 0 = abnormal.
# Same file as TensorFlow's official tutorial; if Google's link is blocked we use a GitHub copy.
urls = ["http://storage.googleapis.com/download.tensorflow.org/data/ecg.csv",
        "https://raw.githubusercontent.com/kanesp/ECG_Anomaly-Detection/master/data/ecg.csv"]
for url in urls:
    try:
        df = pd.read_csv(url, header=None)
        print("Loaded from", url)
        break
    except Exception as e:
        print("Could not load", url, "- trying the next link")

X = df.iloc[:, :140].values.astype("float32")
y = (df.iloc[:, 140] == 0).astype(int).values      # we flip it: 1 = ABNORMAL (the thing we want
                                                    to catch)
print("Beats:", X.shape, "  abnormal:", y.sum(), "  normal:", (y == 0).sum())
```

This cell repeats the binary CSV loading and label inversion from Notebook 1 so Notebook 2 can run independently. The same shape and class counts should appear. A successful run uses 140 float32 features per beat and abnormal as class one.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
                                                    stratify=y)

def build_model(batch_size=None):
    return keras.Sequential([
        keras.Input(shape=(140, 1), batch_size=batch_size),
        layers.Conv1D(16, 7, activation="relu"),
        layers.MaxPooling1D(4),
        layers.Conv1D(32, 5, activation="relu"),
        layers.MaxPooling1D(2),
        layers.Flatten(),
        layers.Dense(32, activation="relu"),
        layers.Dropout(0.3),
        layers.Dense(1, activation="sigmoid"),
    ])

keras.utils.set_random_seed(42)
model = build_model()
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.fit(X_train[..., None], y_train, epochs=20, batch_size=32, validation_split=0.1, verbose=0)
keras_acc = model.evaluate(X_test[..., None], y_test, verbose=0)[1]
print(f"Keras model: {model.count_params():,} weights, test accuracy {keras_acc:.1%}")
```

`build_model` returns a newly constructed network each time it is called. Its optional `batch_size` controls the batch dimension in the input specification. During ordinary training, the default permits varying batch lengths.

The split, architecture, seed, loss, optimiser and training settings follow Notebook 1. `model.evaluate(...)[1]` selects the accuracy entry from the returned loss-and-metric sequence. The saved result is 17,121 parameters and 99.3% test accuracy. The printed word “weights” counts biases as well.

## 9.2 Creating a fixed-batch inference model

```

tiny = build_model(batch_size=1)
tiny.set_weights(model.get_weights())

# 1) plain float32 TFLite
converter = tf.lite.TFLiteConverter.from_keras_model(tiny)
tflite_f32 = converter.convert()

# 2) full int8 quantization: needs ~100 example inputs to learn the value ranges
def representative_data():
    for i in range(200):
        yield [X_train[i:i+1, :, None]]

converter = tf.lite.TFLiteConverter.from_keras_model(tiny)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = representative_data
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8
tflite_int8 = converter.convert()

open("ecg_f32.tflite", "wb").write(tflite_f32)
open("ecg_int8.tflite", "wb").write(tflite_int8)
model.save("ecg_model.keras")

sizes = {"Keras file": os.path.getsize("ecg_model.keras"),
         "TFLite float32": len(tflite_f32),
         "TFLite int8": len(tflite_int8)}
for k, v in sizes.items():
    print(f"{k:15s} {v/1024:7.1f} KB")

```

`tiny=build_model(batch_size=1)` creates the same architecture with an explicit one-example batch. Its input shape is (1,140,1). `set_weights(model.get_weights())` copies the learned arrays from the trained model. This preserves its learned numerical parameters without another training run.

The converter serialises a supported inference computation graph into TensorFlow Lite format, often stored with the `.tflite` extension. LiteRT documentation uses the same model format in relevant conversion workflows [10]. A runtime must support the operations, tensor types and quantisation scheme in the converted graph.

Conversion creates an inference representation. Dropout's training-time random masking is inactive during ordinary inference. Optimiser state needed to resume training is not required for this deployment graph.

## 9.3 The float32 conversion

The first converter is constructed from `tiny`, and `convert()` returns a byte sequence. In the floating-point version, numerical tensors largely retain their floating-point representation. A float32 value uses four bytes, or 32 bits.

There are  $17121 \times 4 = 68484$  bytes of float32 parameters, approximately 66.9 KiB. A complete file also stores structure and other metadata. The `.keras` training file can contain architecture, optimiser state and archive information, so its size is not just four times the parameter count.

## 9.4 The representative dataset

Integer calibration needs examples of the actual model input range. The generator `representative_data` yields 200 training beats. The slice `i:i+1` retains the batch dimension, and `[:,None]` in the displayed indexing adds the signal channel, producing the required `(1, 140, 1)` shape.

The generator yields a list containing the model input array because the conversion interface can accept models with multiple inputs. These examples support range estimation for activations. They should represent expected deployment inputs and preprocessing. Using training examples preserves the test set's role in evaluation.

The comment above this code mentions around 100 examples, while the actual loop uses 200. The loop determines what executes. A fixed number of calibration examples does not ensure that all important extremes or conditions are covered.

## 9.5 Requesting full integer conversion

`Optimize.DEFAULT` enables the converter's optimisation pathway. Supplying a representative dataset enables calibration. `TFLITE_BUILTINS_INT8` restricts supported built-in operations to integer-compatible forms, while the two inference-type settings request int8 input and output.

These settings together are more specific than the short conversion snippet on slide 16. Setting optimisation alone does not establish full int8 input, output and operators. If an operation cannot meet the requested restrictions, the conversion needs attention.

The two `open(...,"wb").write(...)` calls write binary model bytes. `model.save` writes the Keras model. `os.path.getsize` reports bytes on disk, and `len(tflite_int8)` reports the length of the converted byte sequence.

# 10 Quantisation explained with numbers

## 10.1 Representing values on an integer grid

An int8 value can take 256 integer values from  $-128$  to  $127$ . Quantisation represents a range of real values using this finite set. For scale  $s > 0$  and integer zero point  $z$ , a common mapping is

$$q = \text{clip}(\text{round}(x/s) + z, -128, 127), \quad \hat{x} = s(q - z).$$

The scale is the real-valued spacing between adjacent integer codes. The zero point is the code corresponding to real zero. Rounding introduces approximation, and clipping handles values outside the representable range.

The notebook computes `round(x/scale + zero)`. With an integer zero point, this expresses the same intended grid apart from possible floating-point boundary details. Implementation rounding conventions should be checked when comparing platforms exactly.

For the saved input scale  $s \approx 0.0439716$  and  $z = 33$ , an input value of  $0.613$  gives

$$q \approx \text{round}(0.613/0.0439716 + 33) = 47,$$

$$\hat{x} = 0.0439716(47 - 33) \approx 0.6156024.$$

The reconstruction differs by about  $0.0026$  in the input's units. Real zero maps to  $33$ . The representable endpoints are approximately  $-7.0794$  and  $4.1333$ . Values beyond them saturate under clipping.

Network weights and activations can have different scales and zero points, and some tensors use per-channel scales. Biases and accumulations can use wider integer types. The input mapping illustrates the principle; the full converter and runtime manage the internal tensor details [10,11].

## 10.2 Why the complete file does not shrink exactly fourfold

```
plt.figure(figsize=(9, 3.5))
bars = plt.barh(list(sizes.keys())[::-1], [v / 1024 for v in list(sizes.values())[::-1]], color
               =["#245BBF", "#8FBOE6", "#5B6376"])
plt.bar_label(bars, fmt="%.1f KB", fontsize=12)
plt.xlabel("size (KB)"); plt.title("Model size - ESP32 flash is 4,096 KB"); plt.show()
```

The chart reverses the dictionary entries so its bars appear in the desired order. Each byte count is divided by 1,024. Strictly, these values are KiB, although the notebook labels them KB.

| Format              | Saved notebook, KiB | Slide value labelled KB |
|---------------------|---------------------|-------------------------|
| Keras training file | 239.3               | 239.4                   |
| TFLite float32      | 70.2                | 70.8                    |
| TFLite int8         | 22.7                | 23.3                    |

The saved integer file contains 23,288 bytes. The ratio  $70.2/22.7 \approx 3.09$  is the observed file-size reduction relative to float32 in this run. Four bytes becoming one describes an individual value's storage reduction. Graph metadata, quantisation parameters and wider stored values affect the complete file.

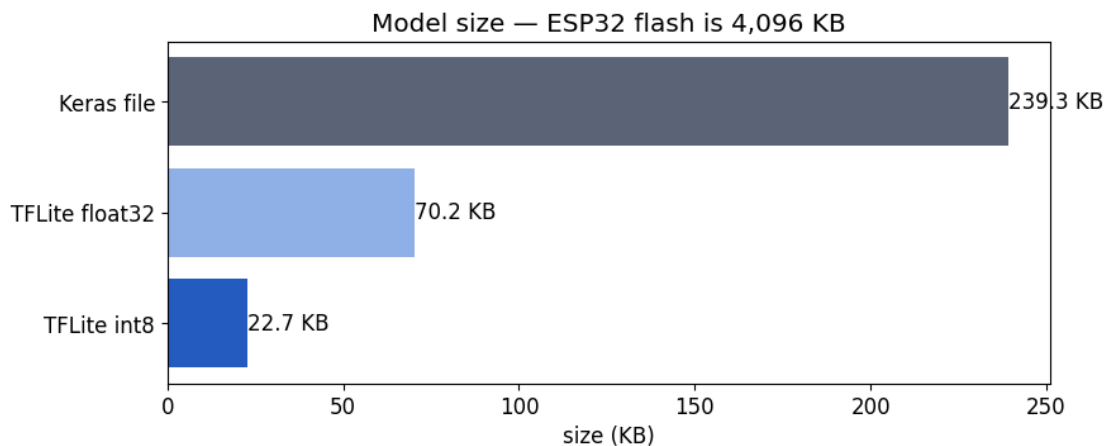


Figure 9: The saved model-size comparison. The graph uses KB to label quantities calculated by dividing bytes by 1,024.

File size indicates that the model bytes are small relative to the example flash capacity. It does not establish that the firmware, operator implementations and working tensors all fit or run fast enough.

## 11 Running and inspecting the converted models

### 11.1 The interpreter loop

```
def run_tflite(model_bytes, X_data):
    interp = tf.lite.Interpreter(model_content=model_bytes)
    interp.allocate_tensors()
    inp = interp.get_input_details()[0]
    out = interp.get_output_details()[0]
    probs = []
    start = time.time()
    for beat in X_data:
        x = beat.reshape(1, 140, 1)
        if inp["dtype"] == np.int8:
            # int8 model: scale the input to
            int8
            scale, zero = inp["quantization"]
            x = np.round(x / scale + zero).clip(-128, 127).astype(np.int8)
            interp.set_tensor(inp["index"], x)
            interp.invoke()
            p = interp.get_tensor(out["index"])[0, 0]
            if out["dtype"] == np.int8:
                # and scale the output back
                scale, zero = out["quantization"]
                p = (float(p) - zero) * scale
            probs.append(p)
    ms = (time.time() - start) / len(X_data) * 1000
    return np.array(probs), ms

for name, m in [("TFLite float32", tflite_f32), ("TFLite int8", tflite_int8)]:
    probs, ms = run_tflite(m, X_test)
    acc = ((probs > 0.5).astype(int) == y_test).mean()
    print(f"{name:15s} accuracy {acc:.1%}    {ms:.3f} ms per heartbeat (on this computer)")
    print(f"'Keras (original)':15s} accuracy {keras_acc:.1%}")
```

The interpreter loads the model bytes and `allocate_tensors()` prepares its tensor storage. Input and output details include tensor indices, shapes, data types and quantisation metadata. The index identifies the buffer used by `set_tensor` or `get_tensor`.

Each waveform is reshaped to (1,140,1). For int8 input, the loop converts through the input scale and zero point, clips to the valid range, and casts to `np.int8`. For float32 input, the original float32 array is retained.

`set_tensor` copies the input into the interpreter. `invoke` runs the graph. `get_tensor` retrieves the output, and `[0,0]` selects the one scalar probability code. If this is int8, subtracting the output zero point and multiplying by its scale recovers the real-valued approximation.

The returned array contains one probability per waveform. Comparing `probs>0.5` with `y_test` produces Booleans; their mean is the fraction correct. The stored Keras, float32 and int8 results all round to 99.3% accuracy. This does not establish identical probabilities or identical mistakes. For a more complete comparison, inspect class metrics, prediction disagreements and probability differences.

### 11.2 Interpreting the timing correctly

The timer starts after allocation and covers the per-example loop, including Python work, input conversion and tensor transfers. It divides by the number of examples to obtain milliseconds per beat.

The saved notebook reports 0.013 ms for float32 and 0.028 ms for int8 on its host computer. The speaker notes record 0.007 and 0.015 ms from another run. Neither is an ESP32 measurement. The int8 path includes additional NumPy conversion work, and runtimes use hardware-specific kernels. File compression does not guarantee speed improvement on every platform.

A target-device benchmark should distinguish model invocation from acquisition and preprocessing. Record the board, clock, runtime, operator kernels and memory arrangement. Include complete pipeline latency when judging whether an application responds in time.

### 11.3 Input metadata and converted operators

```
# What does an int8 model look like inside?
interp = tf.lite.Interpreter(model_content=tflite_int8); interp.allocate_tensors()
print("Input :", interp.get_input_details()[0]["shape"], interp.get_input_details()[0]["dtype"].
      __name__,
      " scale/zero-point:", interp.get_input_details()[0]["quantization"])
print("Operations the chip must support:", sorted({op["op_name"] for op in interp.
      _get_ops_details()} - {"DELEGATE"}))
```

The saved input has shape [1, 140, 1], dtype int8, scale approximately 0.0439716 and zero point 33. The operation list is CONV\_2D, FULLY\_CONNECTED, LOGISTIC, MAX\_POOL\_2D and RESHAPE.

The converter expresses the one-dimensional convolutions and pooling using compatible two-dimensional operations with an extra dimension. This explains why the deployment resolver uses Conv2D although the training model used Conv1D. LOGISTIC implements sigmoid, and FULLY\_CONNECTED implements a dense layer.

The leading underscore in `_get_ops_details()` marks a private API. It is useful for this inspection but can change between versions. `DELEGATE` describes delegated execution and is excluded from the printed underlying operator set. Desktop LiteRT/TFLite and TensorFlow Lite Micro are related runtimes with different deployment capabilities; desktop success alone does not validate the microcontroller build [12].

## 11.4 Exercise: reducing the dense layer

```
# Exercise 1: a smaller dense layer
def build_small(batch_size=None):
    return keras.Sequential([keras.Input(shape=(140, 1), batch_size=batch_size),
                             layers.Conv1D(16, 7, activation="relu"), layers.MaxPooling1D(4),
                             layers.Conv1D(32, 5, activation="relu"), layers.MaxPooling1D(2),
                             layers.Flatten(), layers.Dense(8, activation="relu"), layers.Dense(1, activation="sigmoid")])
keras.utils.set_random_seed(42)
small = build_small(); small.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
small.fit(X_train[..., None], y_train, epochs=20, batch_size=32, validation_split=0.1, verbose=0)
small1 = build_small(1); small1.set_weights(small.get_weights())
c = tf.lite.TFLiteConverter.from_keras_model(small1)
c.optimizations = [tf.lite.Optimize.DEFAULT]; c.representative_dataset = representative_data
c.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
c.inference_input_type = tf.int8; c.inference_output_type = tf.int8
small_int8 = c.convert()
p, _ = run_tflite(small_int8, X_test)
print(f"Weights {small.count_params():,} int8 size {len(small_int8)/1024:.1f} KB accuracy {(p > 0.5) == y_test.mean():.1%}")
```

The smaller architecture uses eight hidden dense units and removes dropout. This changes two training choices, so its comparison does not isolate dense width alone. The convolutional outputs still flatten to 448 values.

Its hidden dense layer has  $(448 + 1)8 = 3592$  parameters, and its output has  $(8 + 1)1 = 9$ . Adding 128 and 2,592 convolution parameters gives 6,321 overall. This is 10,800 fewer parameters than the original.

The cell trains a new network, constructs its fixed-batch copy, copies the weights and performs calibration and conversion again. The speaker notes report a 12.1 KB file and 99.3% accuracy for their experiment. The uploaded exercise cell has no saved output. Run it and record your own result, including recall and the new quantisation metadata. The smaller architecture can need different input ranges or working memory after conversion.

## 12 Exporting the model and understanding the ESP32 template

### 12.1 Turning bytes into a C array

```
def to_c_array(data, name="ecg_model"):
    lines = [f"// Heartbeat classifier, int8, {len(data)} bytes", "#include <stdint>", "",
             f"alignas(16) const unsigned char {name}[] = {{"]
    for i in range(0, len(data), 12):
        lines.append(" " + ", ".join(f"0x{b:02x}" for b in data[i:i+12]) + ",")
    lines += ["};", f"const unsigned int {name}_len = {len(data)};"]
    return "\n".join(lines)

header = to_c_array(tflite_int8)
open("ecg_model.h", "w").write(header)
print(header[:600], "\n...")
print("\necg_model.h written:", round(os.path.getsize("ecg_model.h") / 1024, 1), "KB of text")
```

`to_c_array` builds a text representation of the binary bytes. A hexadecimal number such as `0x20` represents one byte, here decimal 32. `:02x` prints two hexadecimal digits. The loop writes 12 values per text line for readability.

`const unsigned char` declares immutable byte values. `alignas(16)` requests a 16-byte-aligned starting address, which helps satisfy runtime alignment requirements. The length constant records how many bytes belong to the model. `"\n ".join(lines)` combines all lines with newline characters.

The saved header holds 23,288 model bytes but occupies about 140.4 KiB as text because each byte is written using several characters, punctuation and spacing. The compiled byte array represents the binary content; its source text length is not the model's binary size. The speaker notes' earlier header contains 23,840 bytes, matching their different conversion output.

### 12.2 Exporting input and output scales

```
# Quantization numbers the ESP32 sketch needs
inp = interp.get_input_details()[0]; out = interp.get_output_details()[0]
print(f"INPUT_SCALE = {inp['quantization'][0]:.8f}; INPUT_ZERO = {inp['quantization'][1]};")
print(f"OUTPUT_SCALE = {out['quantization'][0]:.8f}; OUTPUT_ZERO = {out['quantization'][1]};")

# One test heartbeat, to paste into the sketch for a first test
beat = X_test[0]
print("True label of X_test[0]:", "abnormal" if y_test[0] else "normal")
print("First 10 samples:", ", ".join(f"{v:.3f}" for v in beat[:10]), "...")
np.savetxt("test_beat.txt", beat[None], delimiter=",", fmt="%.3f") # all 140 values, ready
                           to paste into the sketch
```

This cell reads quantisation metadata from the interpreter and prints constants for the sketch. The supplied output is

$$s_{\text{in}} \approx 0.04397160, \quad z_{\text{in}} = 33, \quad s_{\text{out}} = 0.00390625, \quad z_{\text{out}} = -128.$$

Here  $0.00390625 = 1/256$ . An output code of zero reconstructs to  $(0 - (-128))/256 = 0.5$ . A code of 64 reconstructs to 0.75. The maximum code 127 reconstructs to  $255/256 \approx 0.9961$ .

Use metadata from the exact model being deployed. Retraining or reconversion can change it. Reading the runtime tensor's `params.scale` and `params.zero_point` avoids manually copying rounded constants.

`test_beat.txt` stores all 140 values from the first test beat, which is labelled normal in the saved run. The three-decimal formatting rounds the values. For a strict host-to-device parity check, preserve enough precision or export the exact quantised integer input used by the host.

## 12.3 The supplied Arduino sketch

```
%%writefile ecg_esp32.ino
#include <TensorFlowLite_ESP32.h>
#include "tensorflow/lite/micro/micro_mutable_op_resolver.h"
#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/schema/schema_generated.h"
#include "ecg_model.h"

// Paste the numbers printed in the notebook here
const float INPUT_SCALE = 0.0f, OUTPUT_SCALE = 0.0f;
const int INPUT_ZERO = 0, OUTPUT_ZERO = 0;

constexpr int kArenaSize = 20 * 1024; // working memory for the model
alignas(16) uint8_t tensor_arena[kArenaSize];
tflite::MicroInterpreter* interpreter;

float beat[140]; // one heartbeat: fill from the AD8232 sensor

void setup() {
  Serial.begin(115200);
  const tflite::Model* model = tflite::GetModel(ecg_model);
  static tflite::MicroMutableOpResolver<5> resolver; // only the 5 operations our model uses
  resolver.AddConv2D(); resolver.AddMaxPool2D(); resolver.AddReshape();
  resolver.AddFullyConnected(); resolver.AddLogistic();
  static tflite::MicroInterpreter static_interpreter(model, resolver, tensor_arena, kArenaSize);
  interpreter = &static_interpreter;
  interpreter->AllocateTensors();
  Serial.println("Heartbeat model ready");
}

void loop() {
  // 1. read 140 samples around the next R peak into beat[] (not shown)
  TfLiteTensor* input = interpreter->input(0);
  for (int i = 0; i < 140; i++) {
    input->data.int8[i] = (int8_t) round(beat[i] / INPUT_SCALE + INPUT_ZERO);
  }
  interpreter->Invoke();
  int8_t raw = interpreter->output(0)->data.int8[0];
  float p_abnormal = (raw - OUTPUT_ZERO) * OUTPUT_SCALE;
  Serial.printf("P(abnormal) = %.2f %s\n", p_abnormal, p_abnormal > 0.5 ? "<-- CHECK" : "");
  delay(1000);
}
```

The notebook magic `%%writefile` writes the cell body to `ecg_esp32.ino`. It does not compile or flash a board. The remaining code is C++, which the Arduino toolchain can compile when the required board support and compatible library are available.

The include directives make the model schema, micro-interpreter, operation resolver and generated byte array available. `GetModel` interprets the stored model structure. A robust application also checks the model schema version against the runtime.

`MicroMutableOpResolver<5>` reserves room for five operation registrations. The five `Add...` calls match the converted operation list. Registering needed operations helps control code inclusion. Exact constructor signatures and available kernels depend on the library version, so use the installed

library's working examples as a compatibility reference.

The arena is a 20 KiB global buffer. The interpreter object inside `setup` is `static`, so its lifetime continues after `setup` returns. The global pointer refers to that object. `AllocateTensors` prepares the model's working storage; its returned status needs checking before inference can proceed.

## 12.4 What must be completed before using the sketch

The source is explicitly a template. Its input and output scales begin at zero. Dividing by `INPUT_SCALE` in that state is invalid. The global `beat[140]` is zero-initialised and is never filled by acquisition code. The comment describes the missing step.

After installing a compatible library, a sensible first test supplies a known prepared beat, checks allocation and invocation status, verifies input shape and type, and compares the device output with the host model. The provided material does not contain a successful hardware test or a measured arena requirement.

The template's `int8` cast also omits clipping. A safer conversion fragment uses the loaded tensor's metadata and saturates before the cast:

```
TfLiteTensor* input = interpreter->input(0);
float scale = input->params.scale;
int zero = input->params.zero_point;
// First verify that input is int8, has 140 elements,
// and that scale is finite and greater than zero.
for (int i = 0; i < 140; ++i) {
    float q = roundf(beat[i] / scale) + zero;
    q = fminf(127.0f, fmaxf(-128.0f, q));
    input->data.int8[i] = static_cast<int8_t>(q);
}
if (interpreter->Invoke() != kTfLiteOk) {
    Serial.println("Inference failed");
    return;
}
TfLiteTensor* output = interpreter->output(0);
float p_abnormal =
    (int(output->data.int8[0]) - output->params.zero_point)
    * output->params.scale;
```

This is an explanatory fragment, not a completed acquisition program. It assumes the surrounding code has validated tensor types and metadata, included the needed mathematical declarations, and filled the input with finite values. Checking `AllocateTensors() != kTfLiteOk` belongs in `setup` before the loop can use the tensors.

## 12.5 The missing link between the sensor and the model

The sensor produces an analogue waveform that the ADC samples. The training file contains prepared, equal-length beats with its own scaling and interpolation. A device pipeline must reproduce the relevant segmentation, alignment, resampling and normalisation before sending values to the model.

Collecting any 140 ADC samples does not establish that correspondence. At 360 samples per second, 140 raw samples span approximately 0.389 seconds, while the dataset's 140 positions describe an interpolated beat representation. The training transformation must be documented and reproduced, or the model must be trained using the exact device pipeline.

Continuous acquisition also needs timing, buffering and signal-quality handling. The template's

`delay(1000)` pauses its main loop; it cannot serve as the complete sampling scheduler for a 360 Hz stream. Capturing samples after an R peak may require waiting for part of the beat, which contributes to latency.

## 12.6 Raspberry Pi inference and file download

```
from ai_edge_litert.interpreter import Interpreter
interp = Interpreter(model_path="ecg_int8.tflite")
interp.allocate_tensors()
```

On a compatible Pi installation, the file-based interpreter can load the converted model. The same input shape, quantisation and output reconstruction principles apply. Package wheels, operating system and processor architecture need to match. Copying a model to the Pi does not automatically connect the ECG sensor or implement preprocessing.

A camera and suitable software can also run object detection on a Pi. Number-plate reading and crop-disease recognition need models and processing appropriate to those tasks; the general detector from Day 4 does not already provide every specialised capability. ROS can connect robot software components, and a microcontroller can handle time-sensitive motor or sensor tasks alongside the Pi.

```
# Download the files to your computer (Colab only)
try:
    from google.colab import files
    for f in ["ecg_int8.tflite", "ecg_model.h", "ecg_esp32.ino"]:
        files.download(f)
except Exception:
    print("Not in Colab - the files are in the current folder:", [f for f in os.listdir(".") if f.startswith("ecg")])
```

In Colab, the loop requests downloads of the integer model, header and sketch. Elsewhere, the exception block lists matching files in the working directory. The broad exception can also catch a failed download, so inspect any underlying issue. `test_beat.txt` is created earlier but is not included in this download loop; download it separately when using the reference-beat test.

## 13 Robotics and learning from rewards

*Material covered: Slides 21–24 and 27; Notebook 3 introduction.*

### 13.1 Sense, think and act

A robot observes its surroundings, chooses actions and changes something in the physical world. Sensors can include cameras, lidar, ultrasonic range sensors, inertial sensors and wheel encoders. An encoder provides information about shaft or wheel movement. Lidar estimates distances using light, while an ultrasonic sensor uses sound.

The computation stage may estimate position, build a map, recognise an object or choose a route. Actuators such as motors and grippers execute commands. Their actions alter the next observations, creating a feedback loop.

A proportional-integral-derivative controller, or PID controller, adjusts a command using present error, accumulated error and the rate of error change. For a line follower, the error could describe

how far the line is from the desired image position. Classical control and learned perception can operate within the same robot.

SLAM means simultaneous localisation and mapping: estimating where the robot is while building a representation of its environment. SLAM includes geometric and probabilistic approaches as well as learned components. It is not automatically a machine-learning algorithm. ROS provides software interfaces and communication infrastructure for robotics components; it does not itself supply a complete learned behaviour.

## 13.2 The reinforcement-learning vocabulary

The **agent** chooses actions. The **environment** responds with a new situation and a reward. A **state** represents information used for decision-making. An **action** is one available choice. A **reward** is a numerical feedback signal. A **policy** specifies how the agent selects an action from the available information.

An **episode** is one sequence of interaction from a starting situation to an ending condition or time limit. In the warehouse, an episode begins at the starting square and ends when the goal is reached or the step budget is exhausted.

The reward is designed by the environment's author. It expresses what the training procedure should favour. The agent learns from experience about the consequences of its choices. A poorly chosen reward can encourage an unwanted shortcut, so reward design and evaluation matter.

The return is a sum of future rewards, often discounted:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$$

The discount  $\gamma$  controls how later rewards contribute. With  $\gamma = 0.95$ , a reward delayed one additional step receives another factor of 0.95. The rewards themselves remain those defined by the environment; discounting changes their contribution to the return.

# 14 Notebook 3: building the warehouse environment

## 14.1 Setup and state encoding

```
!pip install -q gymnasium
```

Gymnasium supplies the taxi environment later in the notebook. The small warehouse is implemented directly in Python and NumPy.

```
import numpy as np
import matplotlib.pyplot as plt
import time
from IPython.display import clear_output
plt.rcParams["font.size"] = 12
print("Ready!")
```

NumPy supplies the grid, Q-table and random number generators. Matplotlib displays learning curves. `clear_output` and `time.sleep` support the later taxi animation. Their role is presentation, and they are not part of the learning rule.

```

grid = np.array([[0, 0, 0, 0, 0],
                 [0, 1, 1, 0, 0],
                 [0, 0, 0, 1, 0],
                 [1, 1, 0, 1, 0],
                 [0, 0, 0, 0, 2]])      # 0 = floor, 1 = shelf, 2 = goal

N = 5
moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
arrows = ["up", "down", "<-", "->"]

def step(state, action):
    """The environment: given where the robot is and what it does, return (new_state, reward,
    done)."""
    r, c = divmod(state, N)
    dr, dc = moves[action]
    nr, nc = r + dr, c + dc
    if not (0 <= nr < N and 0 <= nc < N) or grid[nr, nc] == 1:
        return state, -5, False      # bump! stay where you are
    new_state = nr * N + nc
    if grid[nr, nc] == 2:
        return new_state, 100, True    # delivered!
    return new_state, -1, False

def show(state=None, path=()):
    for r in range(N):
        row = ""
        for c in range(N):
            s = r * N + c
            if s == state: row += " R "
            elif grid[r, c] == 1: row += " # "
            elif grid[r, c] == 2: row += " G "
            elif s == 0: row += " S "
            elif s in path: row += " . "
            else: row += " . "
        print(row)

show(state=0)

```

The grid has five rows and five columns. Zero indicates free floor, one indicates a shelf, and two indicates the goal. Row zero is at the top and column zero at the left. Each position is encoded by

$$s = 5r + c.$$

Thus  $(r, c) = (2, 3)$  corresponds to state 13. `divmod(13, 5)` returns quotient 2 and remainder 3, recovering the row and column. There are 25 numerical position codes, although shelf positions are not valid places for the robot to occupy.

`moves` stores changes in row and column: up  $(-1, 0)$ , down  $(1, 0)$ , left  $(0, -1)$  and right  $(0, 1)$ . An action is the index of one of these pairs.

## 14.2 Understanding the transition function

`step(state, action)` decodes the position, adds the selected displacement and checks the candidate position. If it is outside the grid or is a shelf, the robot stays in place and receives  $-5$ . Python's `or` short-circuit behaviour prevents the shelf lookup when the boundary test already detects an invalid position.

Reaching the goal returns the new state, reward  $+100$  and `True` for completion. An ordinary valid move receives  $-1$  and `False`. The terminal move receives  $+100$  as its complete reward; the code does not additionally subtract the ordinary move penalty.

The `show` function prints symbols for a robot, shelf, goal, start, route or empty square. Its `if/elif` order sets which symbol takes priority. The argument `path=()` defaults to an empty tuple, so no route positions are highlighted unless supplied.

### 14.3 The random baseline

```
rng = np.random.default_rng(1)
state, total, steps = 0, 0, 0
for steps in range(1, 101):
    state, reward, done = step(state, rng.integers(4))
    total += reward
    if done: break
print(f"Random robot: {steps} moves, total reward {total}, delivered: {done}")
```

`default_rng(1)` creates a seeded generator. `rng.integers(4)` selects a uniformly random integer from zero through three. The loop allows at most 100 actions, adds each reward and stops early if the goal is reached.

The saved run reaches the goal in 36 actions with total reward 9. The independent check for these notes reproduced that result. One successful random route does not establish reliable navigation. A distribution across starting conditions and random seeds gives a fuller baseline.

The reward also reveals the effect of collisions. Let  $b$  be the number of bumps among the 35 nonterminal actions. Its total is

$$100 - (35 - b) - 5b = 65 - 4b.$$

Setting this equal to 9 gives  $b = 14$ . The run therefore contains 14 penalised bumps under this environment's rules.

## 15 Q-learning, one update at a time

### 15.1 What the Q-table stores

A Q-value estimates the discounted return associated with taking a particular action in a particular state and then continuing according to the learned decision rule. The table has 25 rows and four columns, so it stores 100 values. Initially every entry is zero.

The update is

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[ r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

Here  $s$  is the current state,  $a$  the chosen action,  $r$  the observed reward,  $s'$  the resulting state,  $\alpha$  the learning rate and  $\gamma$  the discount. The maximum selects the largest current estimate among the next state's actions. At a terminal state, future return is set to zero.

The bracketed quantity is a temporal-difference error: the difference between the one-step target and the existing estimate. Multiplying by  $\alpha$  makes a partial adjustment. This target uses the table's own current estimates, a process called bootstrapping.

## 15.2 A numerical update

Suppose  $Q(s, a) = 10$ , the move gives reward  $-1$ , and the best next-state  $Q$ -value is 20. With  $\alpha = 0.1$  and  $\gamma = 0.95$ ,

$$\text{target} = -1 + 0.95(20) = 18,$$

$$Q_{\text{new}} = 10 + 0.1(18 - 10) = 10.8.$$

If the action instead reaches the goal with reward 100, the target is 100 because the episode ends. Starting from an old value of zero, that first update gives 10. Repeated experience pushes the estimate closer to its target.

## 15.3 Exploration and the training loop

```
Q = np.zeros((N * N, 4))
alpha, gamma = 0.1, 0.95
rng = np.random.default_rng(0)
episode_rewards = []

for episode in range(500):
    epsilon = max(0.05, 1 - episode / 300)      # explore a lot at first, less later
    state, total = 0, 0
    for t in range(100):
        if rng.random() < epsilon:
            action = rng.integers(4)             # explore
        else:
            action = int(np.argmax(Q[state]))     # exploit
        new_state, reward, done = step(state, action)
        best_next = 0 if done else np.max(Q[new_state])
        Q[state, action] += alpha * (reward + gamma * best_next - Q[state, action])
        state, total = new_state, total + reward
        if done: break
    episode_rewards.append(total)

print("Average reward, first 50 episodes:", np.mean(episode_rewards[:50]))
print("Average reward, last 50 episodes: ", np.mean(episode_rewards[-50:]))
plt.figure(figsize=(12, 4))
plt.plot(episode_rewards, alpha=0.4, label="each episode")
plt.plot(np.convolve(episode_rewards, np.ones(20) / 20, mode="valid"), linewidth=3, label="20-episode average")
plt.xlabel("episode"); plt.ylabel("total reward"); plt.legend(); plt.title("The robot learns");
plt.show()
```

An epsilon-greedy policy chooses a random action with probability  $\epsilon$  and an action with the largest  $Q$ -value otherwise. Random selection may still choose the currently best action. When values are tied, `np.argmax` returns the first maximum, which introduces an action-order tie break.

The schedule starts with  $\epsilon = 1$ . At episode 150 it is 0.5, and it reaches its minimum 0.05 at episode 285. It then remains at that floor. This leaves some exploratory behaviour even late in training.

Each of 500 episodes resets the state to zero and permits up to 100 actions. The selected action produces a transition. `best_next=0 if done else ...` handles the terminal case. One table entry is updated, and state and reward totals advance. Learning uses the maximum next-state estimate even when the behaviour policy explores, which is a defining feature of this  $Q$ -learning update.

The stored average reward improves from  $-83.46$  in the first 50 episodes to  $91.8$  in the last 50. Our independent rerun reproduces these figures. Training rewards retain occasional exploratory mistakes, so they need not equal the reward of the final greedy policy.

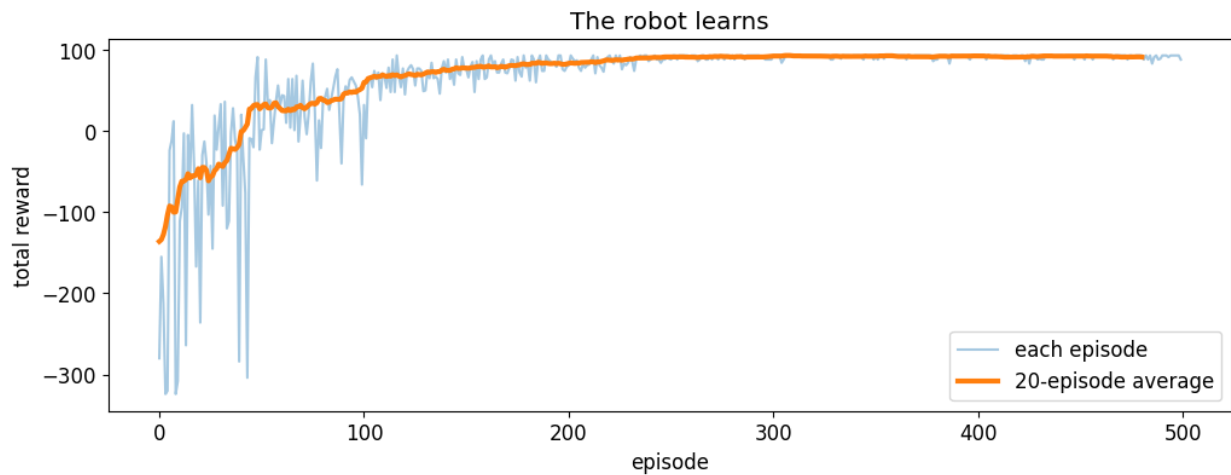


Figure 10: Saved warehouse rewards and their moving average. Individual episodes vary because the agent continues exploring.

`np.ones(20)/20` creates 20 equal averaging weights. `np.convolve(...,mode="valid")` computes averages only where a full 20-episode window fits, producing 481 values from 500 rewards. The plot's default horizontal indices begin at zero for the first window. To label averages by their final episode, plot them against `np.arange(19,len(episode_rewards))`.

## 16 Inspecting what the warehouse agent learned

### 16.1 The greedy route

```
state, path = 0, [0]
for t in range(20):
    state, reward, done = step(state, int(np.argmax(Q[state])))
    path.append(state)
    if done: break
print(f"Trained robot reaches the goal in {len(path) - 1} moves:\n")
show(path=path)
```

This rollout always chooses the largest Q-value and stores each visited state. The path begins with the starting state, so the number of actions is `len(path)-1`. The saved policy reaches the goal in eight moves, which the independent rerun also confirms.

From the top-left to the bottom-right of a five-by-five grid, at least four downward and four rightward moves are required. That gives a lower bound of eight actions. The recorded route achieves it without crossing shelves, establishing shortest length for this start and goal.

The exact saved path is 0, 1, 2, 3, 8, 9, 14, 19, 24. It moves along part of the top row, then through row one to the right edge and down to the goal. This describes the stored route more precisely than saying it uses the entire top row.

Its undiscounted reward is  $7(-1) + 100 = 93$ . Its discounted return is

$$-\sum_{k=0}^6 0.95^k + 100(0.95)^7 \approx 63.8005.$$

The starting Q-value near 64 is consistent with this discounted quantity. The episode plot shows undiscounted totals, so its vertical values differ.

The print statement in the original cell says the goal was reached even if its 20-step loop exhausts without completion. In a reusable evaluation, check `done` and report success separately from path length.

## 16.2 The policy and the value map

```
# The policy: the best arrow in every square
print("Best move in each square:\n")
for r in range(N):
    row = ""
    for c in range(N):
        s = r * N + c
        row += " # " if grid[r, c] == 1 else (" G " if grid[r, c] == 2 else f" {arrows[int(np.
        argmax(Q[s]))]} ")
    print(row)
```

The arrow selected at each free square represents the action with maximum learned Q-value. A policy is a decision rule over states, so this display contains more information than a single route. Shelf and goal squares receive special symbols.

```
# The value of each square = the best Q score there
values = Q.max(axis=1).reshape(N, N)
values[grid == 1] = np.nan
plt.figure(figsize=(6, 5))
plt.imshow(values, cmap="viridis"); plt.colorbar(label="best Q value")
for r in range(N):
    for c in range(N):
        if grid[r, c] != 1:
            plt.text(c, r, f"{values[r, c]:.0f}", ha="center", va="center", color="white",
            fontsize=12)
plt.title("How good is each square? (higher = closer to the goal)"); plt.xticks([]); plt.yticks(
[]); plt.show()
```

`Q.max(axis=1)` selects each row's largest value, giving 25 numbers. Reshaping forms a five-by-five array. Setting shelf entries to `np.nan` leaves them blank in the image. The annotations round values for readability.

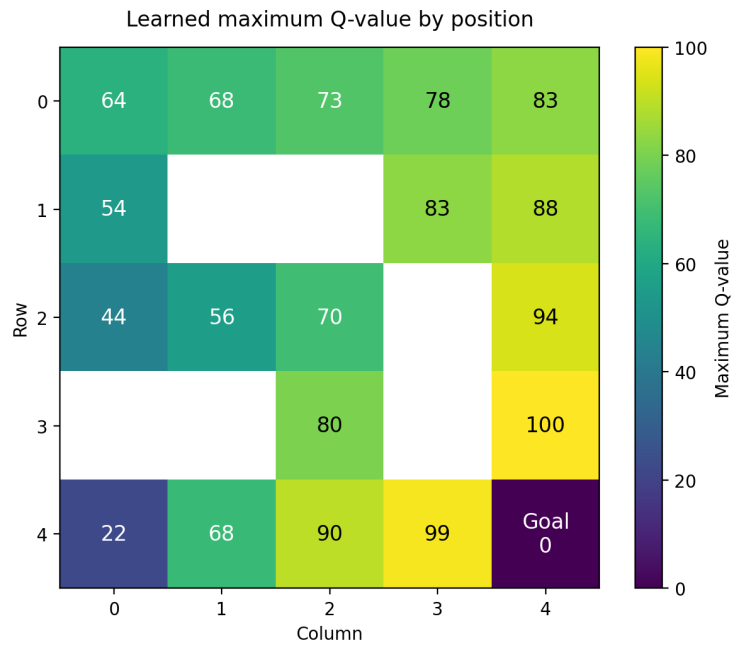


Figure 11: The value map recreated from the independent warehouse rerun. The goal's zero and the low value in the bottom-left require the explanation below.

The goal's row remains zero because an episode ends when it is entered. The reward 100 is received on the transition into that square. The displayed zero therefore represents no subsequent reward after termination.

The original notebook title connects value with closeness to the goal. That is only a rough guide. Values depend on rewards, discounting, accessible routes and how well each state has been learned. The bottom-left square is poorly represented by its low learned value despite having a short route along the bottom row. With a fixed starting state and finite exploration, some positions receive fewer useful updates. The table is a learned estimate, not an exact geometric-distance map.

## 16.3 Exercise 1: changing reward design

```
# Exercise 1: in our run BOTH changes still give the 8-move shortest route.
# Why does it still hurry when moves are free? The discount gamma = 0.95: a +100 reached
# 8 moves from now is worth more than +100 reached 20 moves from now. Try gamma = 1.0 with free
  moves!
def train_grid(bump=-5, move=-1, episodes=500):
    def step2(state, action):
        r, c = divmod(state, N); dr, dc = moves[action]; nr, nc = r + dr, c + dc
        if not (0 <= nr < N and 0 <= nc < N) or grid[nr, nc] == 1: return state, bump, False
        ns = nr * N + nc
        return (ns, 100, True) if grid[nr, nc] == 2 else (ns, move, False)
    Q2 = np.zeros((N * N, 4)); rng = np.random.default_rng(0)
    for ep in range(episodes):
        eps = max(0.05, 1 - ep / 300); s = 0
        for t in range(100):
            a = rng.integers(4) if rng.random() < eps else int(np.argmax(Q2[s]))
            ns, r, d = step2(s, a); Q2[s, a] += 0.1 * (r + 0.95 * (0 if d else Q2[ns].max())) -
            Q2[s, a]; s = ns
            if d: break
        s, n = 0, 0
        for n in range(1, 30):
            s, r, d = step2(s, int(np.argmax(Q2[s])))
            if d: break
        return n
    print("bump -50: route length", train_grid(bump=-50))
    print("free moves: route length", train_grid(move=0))
```

The nested `step2` uses the selected bump and ordinary-move rewards. Each call creates a fresh Q-table and repeats training. Both the stronger bump penalty and free-move variant produce eight-action routes in the independent check.

With free moves and  $\gamma = 0.95$ , a sooner terminal reward still has greater discounted value. An eight-action route contributes  $100(0.95)^7 \approx 69.83$  from the terminal reward; a twenty-action route contributes  $100(0.95)^{19} \approx 37.74$ .

If you also set  $\gamma = 1$  and ordinary moves remain free, successful collision-free routes of different lengths can have the same return. The provided exercise function hard-codes 0.95 in its update, so changing an outer `gamma` variable alone will not alter it.

The function returns a step count without a success flag. Check completion when experimenting with settings that may fail to learn. To compare learning speed, collect reward or success curves across episodes and seeds.

## 17 The taxi environment in Gymnasium

### 17.1 States, actions and rewards

```
import gymnasium as gym
env_id = "Taxi-v4" if "Taxi-v4" in gym.registry else "Taxi-v3" # the name depends on the
library version
env = gym.make(env_id, render_mode="ansi")
state, _ = env.reset(seed=1)
print(env.render())
print("States:", env.observation_space.n, " Actions:", env.action_space.n)
```

Gymnasium gives environments a common interface. The notebook checks the registry for **Taxi-v4** and otherwise requests **Taxi-v3**, accommodating version differences. Record the chosen environment and library version when comparing results.

There are 25 taxi positions, five passenger situations (four waiting locations or inside the taxi), and four destinations. Their product is  $25 \times 5 \times 4 = 500$  encoded states. The six actions are four directions, pickup and drop-off. Their numerical mapping belongs to the taxi environment and differs from the custom warehouse mapping.

`reset(seed=1)` begins an episode and returns an observation plus an information dictionary. `render_mode="ansi"` produces a textual map with formatting codes. The environment's discrete spaces expose their sizes through `.n`.

The basic rewards are +20 for a successful drop-off, −1 for an ordinary step and −10 for an illegal pickup or drop-off [13]. Each value is the complete reward for that type of step.

### 17.2 Evaluating a random driver

```
def evaluate(Q=None, episodes=100):
    """Run 100 test trips. Q=None means a random driver."""
    rewards, successes, steps_list = [], 0, []
    for i in range(episodes):
        s, _ = env.reset(seed=1000 + i)
        total = 0
        for t in range(200):
            a = env.action_space.sample() if Q is None else int(np.argmax(Q[s]))
            s, r, terminated, truncated, _ = env.step(a)
            total += r
            if terminated:
                successes += 1; steps_list.append(t + 1); break
            if truncated: break
        rewards.append(total)
    return np.mean(rewards), successes, (np.mean(steps_list) if steps_list else None)

env.action_space.seed(0)
avg, ok, _ = evaluate(None)
print(f"Random driver: average reward {avg:.0f}, delivered {ok} of 100 passengers")
```

`evaluate` runs episodes with a fixed set of reset seeds, 1000 through 1099 by default. With `Q=None`, it samples actions from the action space. With a table, it chooses the maximum entry for the current state. Seeding the environment and seeding action sampling are separate controls.

`env.step` returns an observation, reward, `terminated`, `truncated` and an information dictionary. Termination represents an ending condition in the task, such as successful delivery. Truncation can

indicate an external limit such as a time cap. Both stop the current rollout.

Success increments only on termination. The mean step count uses successful trips only because failed trips are not appended to `steps_list`. Keep that denominator in mind when comparing agents with different success rates.

The stored random driver succeeds in six out of 100 test trips with average reward around  $-765$ . Many illegal pickup and drop-off attempts produce severe penalties. The result depends on the environment version and action-generator state as well as reset seeds.

### 17.3 Training for 5,000 episodes

```
Q = np.zeros((env.observation_space.n, env.action_space.n))
rng = np.random.default_rng(0)
history = []
start = time.time()
for episode in range(5000):
    s, _ = env.reset(seed=episode)
    epsilon = max(0.05, 1 - episode / 3000)
    total = 0
    for t in range(200):
        a = env.action_space.sample() if rng.random() < epsilon else int(np.argmax(Q[s]))
        s2, r, terminated, truncated, _ = env.step(a)
        best_next = 0 if terminated else np.max(Q[s2])
        Q[s, a] += 0.1 * (r + 0.99 * best_next - Q[s, a])
        s, total = s2, total + r
        if terminated or truncated: break
    history.append(total)
print(f"Trained 5,000 trips in {time.time() - start:.1f} seconds")

plt.figure(figsize=(12, 4))
plt.plot(np.convolve(history, np.ones(100) / 100, mode="valid"))
plt.xlabel("episode"); plt.ylabel("average reward (100 episodes)"); plt.title("Taxi learning curve"); plt.show()

avg, ok, steps = evaluate(Q)
print(f"Trained driver: average reward {avg:.1f}, delivered {ok} of 100 passengers, {steps:.1f} steps per trip")
```

The taxi Q-table has  $500 \times 6 = 3000$  entries. The update uses learning rate 0.1 and discount 0.99. Epsilon reaches its 0.05 floor at episode 2850 under this schedule. Each trip allows up to 200 actions.

The update removes the future-return term only when `terminated` is true. It can bootstrap through a time-limit truncation because the underlying task may continue beyond the imposed observation limit. Treating every truncation as a natural terminal state can change the value target; choose handling consistent with the environment and objective [14].

The saved run takes 7.3 seconds, while the slides and speaker notes describe 3.9 seconds from an earlier run. Timing is hardware- and environment-dependent. The saved final evaluation reports 100 successful deliveries, average reward 7.6 and 13.4 actions per trip.

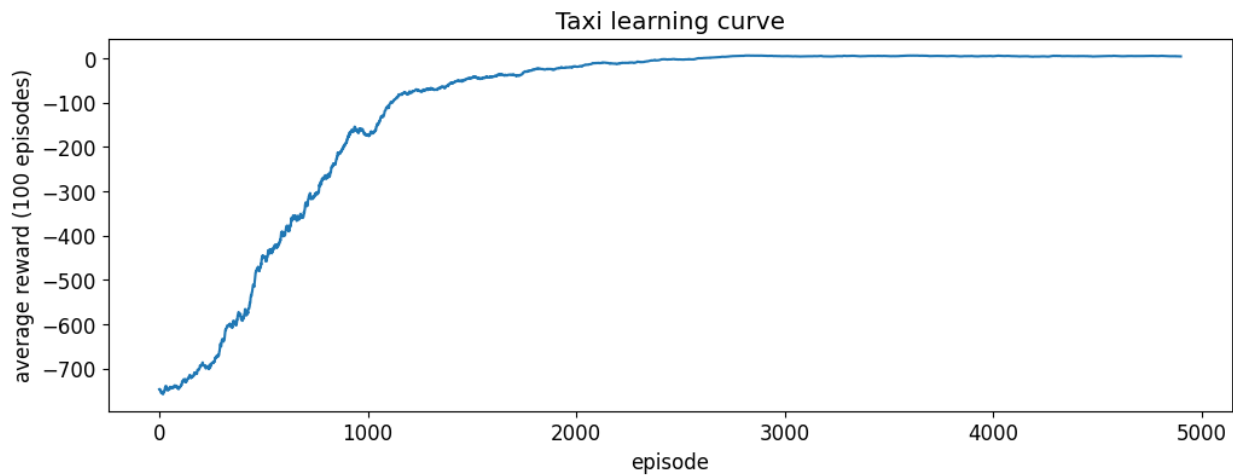


Figure 12: The saved taxi learning curve uses a 100-episode moving average.

For a successful trip with  $L$  actions and no illegal-action penalties, reward is  $20 - (L - 1) = 21 - L$ . At a mean of 13.4 actions, this gives  $21 - 13.4 = 7.6$ , consistent with the stored summary.

The 100 evaluation resets are a small benchmark of a simulated task. Some underlying states can have appeared during training; this is not a claim of transfer to unfamiliar road scenes or physical driving. The phrase “self-driving taxi” names the toy environment.

## 17.4 Watching a trip

```
# Watch one trip
s, _ = env.reset(seed=7)
for t in range(30):
    clear_output(wait=True)
    print(env.render()); print("step", t)
    s, r, terminated, truncated, _ = env.step(int(np.argmax(Q[s])))
    time.sleep(0.4)
    if terminated:
        clear_output(wait=True); print(env.render()); print(f"Passenger delivered in {t + 1} steps! "); break
```

`clear_output(wait=True)` replaces the preceding frame when new output appears. The 0.4-second pause makes movement easy to follow. It is display pacing and is not part of the learned policy. The saved animation ends with delivery in 11 actions.

The loop shown checks termination but does not separately stop on truncation. A general animation loop should handle both. Browser output buffering can also make frames appear late while the underlying state transitions still occur.

## 17.5 Exercise 2: less training

```
# Exercise 2
Q = np.zeros((env.observation_space.n, env.action_space.n)); rng = np.random.default_rng(0)
for episode in range(500):
    s, _ = env.reset(seed=episode); epsilon = max(0.05, 1 - episode / 300)
    for t in range(200):
        a = env.action_space.sample() if rng.random() < epsilon else int(np.argmax(Q[s]))
        s2, r, terminated, truncated, _ = env.step(a)
        Q[s, a] += 0.1 * (r + 0.99 * (0 if terminated else np.max(Q[s2]))) - Q[s, a]; s = s2
        if terminated or truncated: break
    avg, ok, steps = evaluate(Q)
    print(f"500 episodes: average reward {avg:.1f}, delivered {ok} of 100")
```

The exercise resets the table and trains for 500 trips with a faster exploration schedule. The speaker notes report 15 successful deliveries out of 100 and reward  $-168.3$  for their run. No output is stored in this uploaded exercise cell.

The action space's random generator has already been used in preceding cells. Repeating a notebook section without reseeding it can change its trajectory. For a controlled comparison, initialise both the NumPy generator and action-space generator consistently, use the same environment configuration, and repeat across several seeds.

## 17.6 From tables to deep reinforcement learning

A table works when a state can be represented by a small discrete index. Images and continuous sensor vectors create a much larger state space. A neural network can approximate values or policies from these observations.

DQN uses a neural network to approximate action values and was demonstrated on Atari tasks. AlphaGo combined policy and value networks with search and learning procedures. Its system was more elaborate than substituting a network into this notebook's table. These examples illustrate several ways to combine learning with sequential decisions [15,16].

For physical robots, simulation offers a way to collect experience before testing on hardware. Differences in friction, sensing, delays and contact can still affect transfer. Energy control, traffic timing and robot movement all require rewards that reflect the actual task, alongside evaluation of undesirable behaviour.

## 18 Generative AI and next-token prediction

*Material covered: Slides 29–31 and 34–36; Notebook 4 introduction.*

### 18.1 Learning a distribution over possible outputs

A generative model can produce samples such as text, images or audio by learning patterns in data. In this notebook, the output is a sequence of characters forming a name. The model learns probabilities for the next character given the available context.

For a sequence  $x_1, x_2, \dots, x_T$ , its probability can be expressed as

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1}).$$

This reads as a product of next-step probabilities. Generation repeatedly obtains a distribution, chooses an output token, adds it to the context and continues. An end marker tells the generator when to stop.

A token is one unit from a model's vocabulary. Here it is a character. In other language models, tokens can be word pieces, complete words, punctuation or other symbols. The vocabulary size and tokenisation rules depend on the model. One word can occupy several tokens, and a token can also represent punctuation.

The slide's "The capital of Nigeria is" example illustrates a possible distribution over continuations. Its bars are conceptual and were not measured from a specified model. Learning to predict plausible continuations does not give a guarantee that every generated statement is factual.

## 18.2 What this small example teaches

The counting model uses the previous character. The neural model uses the previous three characters. Both produce one next-token distribution at a time. The neural model is a feed-forward network with an embedding layer, following the general neural probabilistic language-model idea described by Bengio and colleagues [18].

This lets us understand representation, probability, loss and sampling without a large training system. Modern language models can add much longer context, different architectures, richer data and additional training stages. Exact model sizes and training datasets are not always public, so the slide's broad frontier-model figures should not be treated as verified specifications for every named product.

## 19 Notebook 4: a counting language model

### 19.1 Loading names and defining a vocabulary

```
import numpy as np
import matplotlib.pyplot as plt
import urllib.request, time
plt.rcParams["font.size"] = 12

url = "https://raw.githubusercontent.com/karpathy/makemore/master/names.txt"
words = urllib.request.urlopen(url).read().decode().split()
print(len(words), "names, e.g.", words[:8])
```

The URL points to the name list used by the makemore teaching project [17]. `urlopen(...).read()` returns bytes, `decode()` turns them into text, and `split()` separates whitespace-delimited names. The saved file contains 32,033 entries, beginning with names such as emma and olivia.

This is a particular name list written with a limited character set. The list's contents shape what the model learns. It is not representative of all naming traditions, spellings or diacritics.

```
chars = sorted(set("".join(words)))
stoi = {c: i + 1 for i, c in enumerate(chars)}; stoi["."] = 0      # letter -> number
itos = {i: c for c, i in stoi.items()}                          # number -> letter
V = len(stoi)
print("Vocabulary:", V, "tokens")

counts = np.zeros((V, V))
for w in words:
    seq = ["."] + list(w) + ["."]
    for a, b in zip(seq, seq[1:]):
        counts[stoi[a], stoi[b]] += 1

plt.figure(figsize=(11, 10))
plt.imshow(counts, cmap="Blues")
labels = [itos[i] for i in range(V)]
plt.xticks(range(V), labels); plt.yticks(range(V), labels)
plt.xlabel("next letter"); plt.ylabel("current letter"); plt.title("How often each letter follows another")
plt.show()
```

`"".join(words)` concatenates the names. `set` finds distinct characters and `sorted` gives them a stable order. The dictionary `stoi` maps each letter to an integer starting at one, then assigns the marker `.` to zero. `itos` reverses the mapping. The saved vocabulary contains 27 tokens: 26 letters and the marker.

Token identifiers act as lookup indices. An identifier of 20 does not make a letter twice as meaningful as identifier 10. The model's representations and probability estimates supply the learned relationships.

### 19.2 Counting adjacent pairs

`counts` is a  $27 \times 27$  array. Rows identify the current token and columns the following token. For each name, the code inserts a marker at both ends and counts neighbouring pairs. For emma these are

$$(\cdot, e), (e, m), (m, m), (m, a), (a, \cdot).$$

The pair of slices `seq` and `seq[1:]` aligns each character with its successor. `zip` stops at the shorter sequence, so each adjacent pair appears once.

The dot has two roles in this simple setup. At generation start it represents the beginning context; sampling it later ends the name. The heat map shows counts, so a dark cell means that pair appeared frequently in the list. Comparing raw counts across rows also reflects how often their current characters occur.

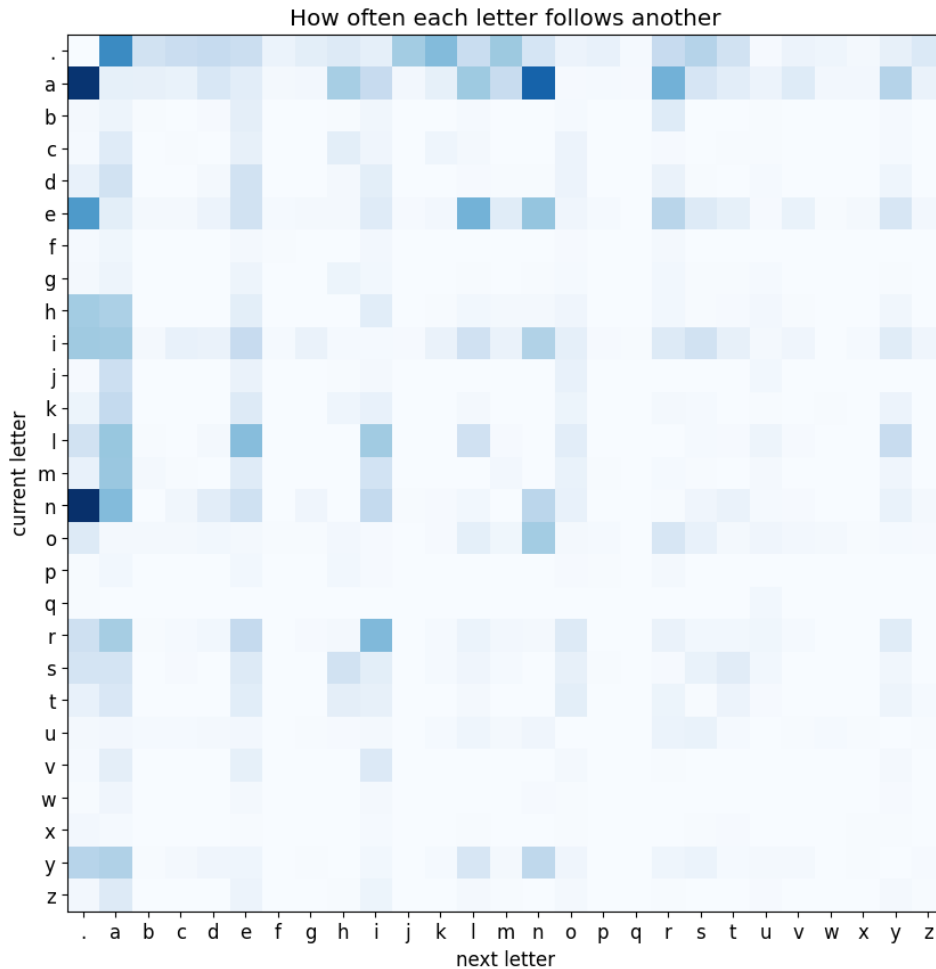


Figure 13: The saved adjacent-character count matrix. Rows are current tokens and columns are next tokens.

## 19.3 From counts to conditional probabilities

```
P = (counts + 1) / (counts + 1).sum(axis=1, keepdims=True)    # turn counts into probabilities
                        (+1 = never exactly 0)

print("Most likely first letters:", [(itos[i], f"{P[0, i]:.1%}") for i in np.argsort(-P[0])
                                     [:5]])
print("Most likely after 'a':      ", [(itos[i], f"{P[stoi['a'], i]:.1%}") for i in np.argsort(-P[
    stoi['a']])[:5]], " ('.' = end)")

rng = np.random.default_rng(42)
def generate_bigram():
    out, i = [], 0
    while True:
        i = rng.choice(V, p=P[i])      # pick the next letter using the probabilities
        if i == 0: return "".join(out)
        out.append(itos[i])
print("\nBigram names:", [generate_bigram() for _ in range(10)])
```

The conversion is

$$P(j | i) = \frac{C_{ij} + 1}{\sum_{k=1}^V (C_{ik} + 1)}.$$

Adding one to every count is Laplace smoothing. It gives unseen pairs a nonzero probability and prevents the logarithm of zero in the later loss calculation. Every row then sums to one.

`sum(axis=1, keepdims=True)` produces one denominator per row while retaining a one-column dimension. NumPy broadcasts those denominators across the matrix columns. `argsort(-P[0])` sorts token indices by decreasing probability after the start marker.

For a miniature vocabulary with three possible successors and counts `[4, 1, 0]`, smoothing gives `[5, 2, 1]`. Dividing by their sum eight gives probabilities `[0.625, 0.25, 0.125]`.

The stored model's most probable first letters are a, k, m, j and s. After a, the end marker has approximately 19.6% probability and n about 16.0%. These are conditional frequencies from this list and smoothing rule.

## 19.4 Sampling a new name

The generator begins with index zero and an empty output list. `rng.choice(V, p=P[i])` samples the next token using the row for the current token. A high-probability token is more likely to be selected, while alternatives can still appear. Sampling the marker returns the accumulated letters as a string.

The fixed random generator seed makes this sequence reproducible if its call history is unchanged. Repeated calls advance the generator. Rerunning only the print line can therefore produce different names from rerunning the whole cell.

Saved outputs include `reslaynn`, `jaylon`, `k` and `siquxxtal`. The model has only one-character context, so it can combine locally common transitions into globally unusual strings. Smoothing also assigns a tiny start-to-end probability, making an empty output possible. The loop has no explicit length cap; a reusable generator can add one.

## 19.5 Measuring next-token loss

```
log_probs = [np.log(P[stoi[a], stoi[b]]) for w in words for a, b in zip(".", w + ".", w + ".")]
bigram_loss = -np.mean(log_probs)
print(f"Random guessing loss: {np.log(27):.3f}")
print(f"Bigram loss: {bigram_loss:.3f}")
```

Each observed adjacent pair selects one conditional probability. Taking its natural logarithm, negating and averaging gives mean negative log-likelihood, also described here as cross-entropy:

$$L = -\frac{1}{M} \sum_{m=1}^M \log P(\text{observed next token}_m \mid \text{context}_m).$$

A probability of 0.5 gives loss about 0.693, while 0.01 gives about 4.605. A model receives a larger penalty when it assigns a low probability to the continuation that actually occurred.

Uniform probabilities over 27 tokens give  $-\log(1/27) = \log 27 \approx 3.296$ . The saved bigram loss is 2.455. Natural logarithms measure this loss in nats per predicted token. The corresponding perplexity,  $\exp(L)$ , is approximately 11.65 for the bigram and 27 for the uniform model. Perplexity summarises uncertainty through this transformation; it is not a percentage accuracy.

This bigram loss is evaluated on the same list used to count pairs. It measures fit to those observed transitions. A fair held-out comparison should construct the counts from training data only and compute loss on a separately reserved set.

## 20 Building the neural name model

### 20.1 Creating contexts and targets

```
block = 3
X, Y = [], []
for w in words:
    context = [0] * block
    for ch in w + ".":
        X.append(context); Y.append(stoi[ch])
        context = context[1:] + [stoi[ch]]
X, Y = np.array(X), np.array(Y)
print("Training examples:", len(X))
for i in range(6):
    print("".join(itos[j] for j in X[i]), "->", itos[Y[i]])
```

`block=3` sets the context length. For each name, the initial context contains three marker indices. Each loop iteration stores that context in `X`, stores the next character's index in `Y`, then drops the oldest context entry and appends the new one.

For `emma`, the examples are

| Input context | Target token |
|---------------|--------------|
| ...           | e            |
| . . e         | m            |
| . em          | m            |
| emm           | a            |
| mma           | .            |

The list expression `context[1:]+[stoi[ch]]` creates a new list. Earlier appended contexts therefore retain their values. Reusing and mutating one list in place would require more care to avoid accidental shared references.

Every name contributes its number of letters plus one end-marker prediction. The stored total is 228,146 examples. `X` has shape (228146, 3) and `Y` has shape (228146,). Each input value is a token identifier.

## 20.2 Splitting examples and defining the network

```
import keras
from keras import layers
keras.utils.set_random_seed(42)

idx = np.random.default_rng(0).permutation(len(X))
split = int(0.9 * len(X)); tr, va = idx[:split], idx[split:]

lm = keras.Sequential([
    keras.Input(shape=(block,)),
    layers.Embedding(V, 10),          # 27 letters x 10 numbers each
    layers.Flatten(),
    layers.Dense(200, activation="tanh"),
    layers.Dense(V),                 # a score for each of the 27 possible next letters
])
lm.compile(optimizer=keras.optimizers.Adam(0.003),
            loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True))
print("Weights:", lm.count_params())

start = time.time()
h = lm.fit(X[tr], Y[tr], validation_data=(X[va], Y[va]), epochs=20, batch_size=256, verbose=0)
print(f"Trained in {time.time() - start:.0f} s")

plt.figure(figsize=(10, 4))
plt.plot(h.history["loss"], label="training"); plt.plot(h.history["val_loss"], label="validation")
plt.axhline(bigram_loss, color="grey", ls="--", label=f"bigram ({bigram_loss:.2f})")
plt.xlabel("epoch"); plt.ylabel("loss"); plt.legend(); plt.title("Neural model beats counting");
plt.show()
print(f"Neural model validation loss: {h.history['val_loss'][-1]:.3f}")
```

A seeded permutation shuffles example indices. The first  $\lfloor 0.9 \times 228146 \rfloor = 205331$  indices become training examples and the remaining 22,815 become validation examples. The model's random seed controls its initialisation and other supported sources of randomness.

The `Embedding(V,10)` layer contains a table with one row of ten learned values per token. Looking up three token indices gives a  $3 \times 10$  array for each example. Flattening concatenates it into 30 values. A 200-unit hidden layer applies an affine transformation and tanh, followed by a final dense layer that produces 27 logits.

| Layer        | Output per example | Parameters           |
|--------------|--------------------|----------------------|
| Token input  | 3 indices          | 0                    |
| Embedding    | $3 \times 10$      | $27 \times 10 = 270$ |
| Flatten      | 30                 | 0                    |
| Dense tanh   | 200                | $(30 + 1)200 = 6200$ |
| Dense logits | 27                 | $(200 + 1)27 = 5427$ |
| Total        |                    | 11,897               |

Embedding values are learned through the same training objective as the other parameters. The token index selects a row, and gradients from prediction errors adjust the selected representations and connected weights. No manually assigned vowel or consonant category is supplied.

### 20.3 Logits, cross-entropy and optimisation

The output layer has no softmax activation. It produces unrestricted scores called logits. `SparseCategoricalCrossentropy(from_logits=True)` tells the loss function to apply the appropriate probability normalisation internally using stable calculations. The targets are integer token indices, which is why the sparse form is appropriate.

Adam uses learning rate 0.003. The model trains for 20 epochs with batches of 256. Each epoch has  $\lceil 205331/256 \rceil = 803$  updates, including a final incomplete batch. The named validation arrays are evaluated after fitting steps without being used directly for gradient updates.

The saved run reports 49 seconds and final validation loss 2.156. The speaker notes describe 31 seconds from a different run. Training time depends on hardware and runtime. A loss of 2.156 corresponds to perplexity approximately 8.64 under this evaluation.

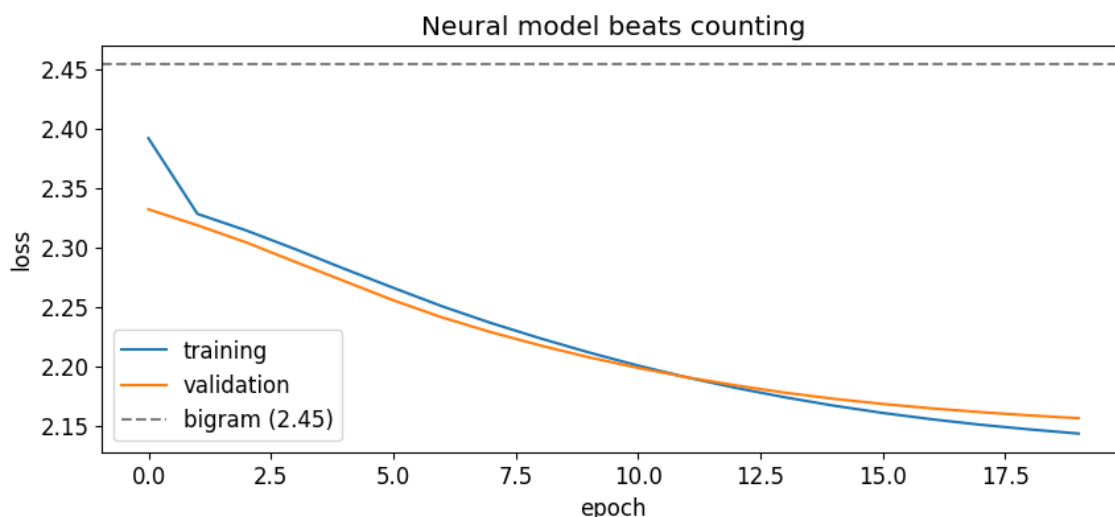


Figure 14: Saved training and validation losses. The bigram line is evaluated on the full name list, which affects how the comparison should be interpreted.

The chart begins its epoch index at zero. Its first plotted point represents the result after the first completed epoch. Training and validation loss both decline in the saved example, indicating improvement under the selected splits.

### 20.4 What this validation split can tell us

The split occurs after turning every name into context-target examples. Examples from the same name can appear in both partitions, and identical short contexts occur throughout the dataset. This evaluates held-out prediction examples from the same collection. It does not isolate completely unseen names.

The bigram was counted using all names, including those contributing validation contexts. Its loss is also computed over the full list. The lower neural validation loss is a useful classroom observation, but the two values do not come from a clean shared held-out comparison.

For a stronger experiment, deduplicate and split complete names first. Fit both models on training names, tune on validation names, and compare losses on the same reserved test-name targets.

## 21 Generating names and inspecting embeddings

### 21.1 The generation function

```
known = set(words)
def generate(temperature=1.0, n=10, seed=1):
    r = np.random.default_rng(seed)
    names = []
    for _ in range(n):
        context, out = [0] * block, []
        while len(out) < 20:
            logits = lm.predict(np.array([context]), verbose=0)[0] / temperature
            p = np.exp(logits - logits.max()); p /= p.sum()
            i = r.choice(V, p=p)
            if i == 0: break
            out.append(itos[i]); context = context[1:] + [i]
        names.append("".join(out))
    return names

names = generate(1.0)
print("Neural model names:", names)
print("Brand-new (not in the training list):", sum(n not in known for n in names), "of", len(
    names))
```

The function creates a local random generator from its `seed` argument, so calling it again with the same settings starts from the same random state. For each requested name, it resets the context to three markers and the output list to empty.

The model receives a batch with one context. `predict(...)[0]` selects its 27 logits. Dividing by temperature adjusts the distribution before softmax. Subtracting the largest logit before exponentiation preserves the resulting probabilities while reducing numerical overflow risk.

After normalising by the exponential sum, `choice` samples a token. A marker ends the name. Otherwise, the letter is appended and the context window advances. The 20-character limit prevents an indefinitely long output. If that cap is reached, the string ends because of the limit, even if no end marker was sampled.

The saved ten-name batch includes **kyeth**, **juvice**, **keanna** and **avorea**. Nine outputs are absent from `known`, the set of supplied names. This is a membership check against that list. A string absent from the list could still be a real person's name, and a generated string's novelty does not establish that it is meaningful or culturally appropriate.

### 21.2 What the embedding plot shows

```
# Look inside: the letter embeddings (first 2 of the 10 numbers)
E = lm.layers[0].get_weights()[0]
plt.figure(figsize=(7, 7))
plt.scatter(E[:, 0], E[:, 1], s=250, c="#FCE3CB", edgecolors="#A84E0F")
for i in range(V):
    plt.text(E[i, 0], E[i, 1], itos[i], ha="center", va="center", fontsize=13)
plt.title("Letter embeddings (2 of 10 dimensions)"); plt.show()
```

`lm.layers[0]` refers to the embedding layer in this Sequential model, since the input specification is not a separate trained layer in that list. `get_weights()[0]` retrieves its  $27 \times 10$  learned table. The scatter plot uses columns zero and one as coordinates and labels each point with its token.

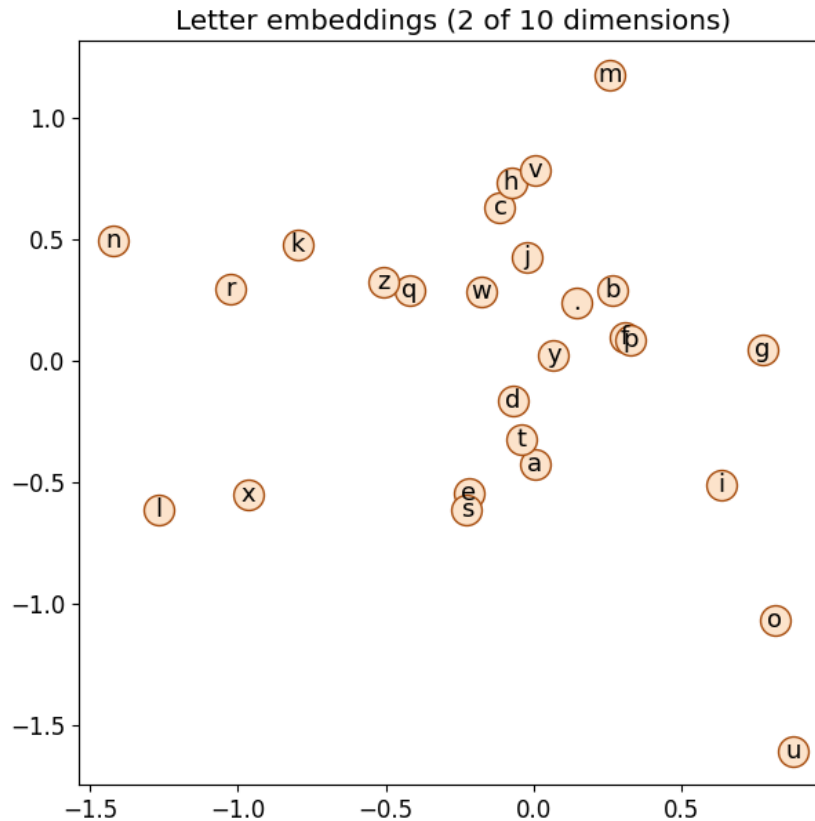


Figure 15: Two coordinates of the saved ten-dimensional embedding table.

In the supplied projection, i, o and u appear toward one side. This is a visual observation from two coordinates. Distances in that plane do not fully describe similarity in ten dimensions, and retraining can rotate or otherwise change representations while preserving useful predictions. The network learns numbers that help its loss objective; a neat linguistic interpretation is not guaranteed for each coordinate.

## 22 Temperature and sampling behaviour

### 22.1 The mathematical effect

For logits  $z_1, \dots, z_V$  and positive temperature  $T$ ,

$$p_i(T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}.$$

A lower positive temperature increases the relative weight of larger logits. A higher temperature makes the distribution flatter. At  $T = 1$ , it is the distribution implied by the unscaled model logits.

For logits  $[2, 1, 0]$ , the probabilities are approximately

| Temperature | First token | Second token | Third token |
|-------------|-------------|--------------|-------------|
| 0.5         | 0.867       | 0.117        | 0.016       |
| 1.0         | 0.665       | 0.245        | 0.090       |
| 2.0         | 0.506       | 0.307        | 0.186       |

The ordering is unchanged, but the strength of preference changes. As positive temperature approaches zero, probability concentrates on the largest logit or tied maxima. At very large temperature, the distribution approaches uniform over finite logits.

### 22.2 Reading the supplied experiment

```
for T in [0.5, 1.0, 1.5]:
    names = generate(T)
    print(f"T = {T}:  new {sum(n not in known for n in names)}/10 ", names)
```

The saved batches have five names absent from the list at  $T = 0.5$ , nine at  $T = 1$ , and eight at  $T = 1.5$ . The count does not increase monotonically in this ten-sample experiment. Sampling variability and the model's distribution both affect it.

The slide's descriptions "safe" and "creative" are informal. Temperature adjusts sampling concentration. A low temperature can repeatedly generate an incorrect answer if the model assigns it high probability. Factuality, originality, usefulness and safety each need their own assessment.

Because `generate` recreates its generator from the same default seed, each temperature experiment starts with matching random-number state. The distributions differ, so sampled tokens and eventual name lengths can still diverge.

### 22.3 Exercise 1: extreme settings

```
# Exercise 1
print("T = 0.1:", generate(0.1))  # almost always the same few very common names
print("T = 3.0:", generate(3.0))  # nearly random letters
```

At 0.1, the generator strongly favours high-logit characters and tends to repeat common patterns. At 3.0, lower-scoring characters gain more probability, often creating unusual strings. The exercise cells have no saved output in the uploaded notebook; the speaker notes describe repeated names such as ana and maria at low temperature.

Use  $T > 0$  for this implementation. Setting zero causes division by zero. Implement deterministic greedy decoding by explicitly selecting `argmax`.

## 22.4 Exercise 2: extending the context

```
# Exercise 2: longer context (block = 5) - usually a slightly lower validation loss
def make_xy(b):
    Xb, Yb = [], []
    for w in words:
        ctx = [0] * b
        for ch in w + ".":
            Xb.append(ctx); Yb.append(stoi[ch]); ctx = ctx[1:] + [stoi[ch]]
    return np.array(Xb), np.array(Yb)
X5, Y5 = make_xy(5)
keras.utils.set_random_seed(42)
lm5 = keras.Sequential([keras.Input(shape=(5,)), layers.Embedding(V, 10), layers.Flatten(),
                        layers.Dense(200, activation="tanh"), layers.Dense(V)])
lm5.compile(optimizer=keras.optimizers.Adam(0.003), loss=keras.losses.
             SparseCategoricalCrossentropy(from_logits=True))
h5 = lm5.fit(X5[tr], Y5[tr], validation_data=(X5[va], Y5[va]), epochs=20, batch_size=256,
            verbose=0)
print(f"block=3 val loss {h.history['val_loss'][-1]:.3f}    block=5 val loss {h5.history['val_loss'][-1]:.3f}")
```

`make_xy(5)` repeats the dataset construction with five context positions. Every name still contributes the same number of next-token targets, in the same order, so the earlier `tr` and `va` index arrays remain aligned.

Five embeddings flatten to 50 values. The hidden layer now has  $(50 + 1)200 = 10200$  parameters, adding 4,000 to the total. The new count is 15,897. The embedding and output-layer parameter counts remain 270 and 5,427.

The speaker notes report validation loss 2.074 for their five-character experiment, compared with 2.156 for the three-character model. There is no saved output for this exercise cell. Longer context supplies more information and increases the model's capacity here, but improvement is an empirical question. The earlier example-level split limitation still applies.

Changing the outer `block` value requires rebuilding examples and a compatible model. Simply changing the variable while using an existing three-input model creates a shape mismatch. The exercise deliberately creates `lm5` separately; the earlier `generate` function still calls `lm` unless you adapt it.

## 23 From the name model to larger generative systems

### 23.1 Attention and the training stages

A Transformer can combine information across many context positions using attention. Queries and keys produce compatibility scores, softmax normalises those scores, and values supply information for weighted combinations. Positional information helps represent order. In causal language modelling, masking prevents a training position from using future target tokens.

Pretraining commonly teaches a language model through next-token prediction on large text collections. Instruction tuning uses examples of requests and desired responses to train a more useful interaction pattern. Preference-based training can further adjust behaviour according to comparisons or other feedback. Different systems use different combinations of methods.

In a common RLHF approach, humans compare responses, a reward model learns from those preferences, and reinforcement learning adjusts a policy using that signal [19]. Other preference-optimisation methods use different procedures. A language-model policy over token sequences is vastly larger than the taxi's 500-state table, even though both can involve reward-guided optimisation.

The slide's GPT-2 entry describes a historical scale comparison. Present-day product names do not reveal exact parameter counts, training data or training methods. The core lesson is how representation, context and training objectives extend the small example.

## 23.2 The optional Colab AI cell

```
try:
    from google.colab import ai
    reply = ai.generate_text("In 3 short bullet points, explain to a Nigerian engineering
                             student "
                             "how a large language model generates text.")
    print(reply)
except Exception as e:
    print("This cell only works in Google Colab (", type(e).__name__, ")")
```

This imports a Colab-specific interface and asks it to generate a short explanation. The service runs an existing model; the cell does not train that model. Its availability can depend on the current Colab environment, account and service conditions. A generic Python installation does not provide the Colab runtime interface.

The uploaded notebook includes a saved generated response. The speaker notes say the feature was not tested in their earlier environment. These are different records, so the saved response establishes that text was stored with this notebook, not that the service is guaranteed available to every student now.

The broad exception prints a Colab-related message for any failure in the block. A quota, permission or service error can therefore produce the same fallback message as an unavailable import. Inspect the exception type and current service instructions if the cell fails.

The stored generated answer itself needs correction in two places. Tokens are vocabulary units represented by identifiers before embedding; vectors are numerical representations of those units. Also, the model estimates a next-token distribution. Its scores do not identify an “exact” correct next word. This makes the cell a useful example of why generated explanations should be reviewed.

## 23.3 Other tools mentioned in the slides

Text generation can support drafting, summarisation and code suggestions. Code still needs review and testing against the intended behaviour. Speech systems can perform transcription, synthesis or audio generation, with different input/output tasks.

Diffusion is one family of generative methods that learns to reverse a noise-adding process. A generator starts from noise and performs learned denoising steps, often conditioned on a prompt. Image and audio generation also use other model families, and the architecture of a named commercial product should be verified before assigning it to one category.

Retrieval-augmented generation, or RAG, retrieves relevant passages from a document collection and supplies them as context to a generator. A course-notes assistant might index sections, retrieve material related to a question, generate an answer and identify the supporting passages. Retrieval quality and answer checking still matter; supplying documents does not guarantee faithful use of

them.

An agent combines a model with a loop for selecting and using tools, observing results and choosing further steps. For example, it may search documents, run a calculation and draft a response. The surrounding application defines tool permissions and how consequential actions are handled. The term agent alone does not specify an RL training method.

### 23.4 Exercise 3: checking Nigerian names and meanings

The notebook suggests asking for Yoruba, Igbo or Hausa names and meanings, then checking the response with a knowledgeable speaker. The purpose is to practise verification. A plausible string and a confident explanation may still contain an invented meaning, incorrect spelling or missing tonal information.

Ask for established names when the goal is a reliable factual list. Keep any request for invented names clearly labelled as invention. Record each proposed name, language, spelling, claimed meaning and verification outcome. The small character model trained on today's list has not learned the linguistic and cultural evidence needed to assign trustworthy meanings to Nigerian names.

A hallucination is generated content presented plausibly despite being unsupported or incorrect. Prompt detail can improve relevance, but evidence must still be checked. This includes references, numerical claims and generated code.

## 24 Putting the five days together

### 24.1 What each day contributed

---

| Day | Main contribution                                                                            |
|-----|----------------------------------------------------------------------------------------------|
| 1   | Learning types, datasets, features, labels and the machine-learning workflow.                |
| 2   | NumPy, pandas, exploratory analysis, classification and regression.                          |
| 3   | Comparing and tuning models, clustering, dimensionality reduction and semi-supervised ideas. |
| 4   | Neural-network training, CNNs, transfer learning and computer vision.                        |
| 5   | Signals, embedded inference, reinforcement learning and generative modelling.                |

---

The course includes 14 notebooks: one on Day 1, three on each of Days 2–4, and four today. Its most reusable habit is to connect each reported result to its input data, preprocessing, learning procedure and evaluation conditions.

### 24.2 Choosing a project that can be finished

An ECG study can begin by reproducing a public-data analysis and documenting the limits of its split. An embedded project can begin with parity testing on known model inputs before adding live acquisition. A camera project can begin with one measurable task, such as counting crossings through a defined region. A document assistant can begin with a small set of course notes and questions whose answers can be checked.

Write down the input, output and evaluation before adding complexity. Keep the dataset source, split, preprocessing, package versions and model settings with your code. Report both successful

examples and errors. If a result is simulated or measured only on a host computer, state that clearly.

The slides list Kaggle, Andrew Ng's courses, fast.ai, Hugging Face and TinyML courses as directions for further study, alongside Data Science Nigeria, Google Developer Groups and local study groups. Course availability, fees and programme names can change. Use the provider's current information when choosing a resource.

### 24.3 Using generated assistance while learning

Give a tool the problem, your current attempt and the point you do not understand. Ask it to explain a calculation or help design a check. Then work through the result and test the code yourself. Keep private data and credentials out of prompts unless the environment and data-use rules explicitly permit them. Follow your course's academic-integrity requirements and acknowledge assistance when required.

A useful project report explains what you tried, why you chose it, how you evaluated it and what remains uncertain. That record will help you more than a collection of unexplained accuracy numbers.

## 25 Review questions and worked answers

**Pause and try.** How many samples represent ten seconds at 360 Hz?

There are  $10 \times 360 = 3600$  samples. Each interval between samples is approximately 2.78 ms. A 650,000-sample recording lasts about 30.09 minutes.

**Pause and try.** Can the forward/backward filter be applied to an unlimited live stream without waiting?

The backward pass needs samples later than the point being processed. A live design needs causal filtering or buffered processing with explicitly measured delay. Its response must be validated for the task.

**Pause and try.** A peak gap is 270 samples at 360 Hz. What is the interval-derived rate?

The interval is  $270/360 = 0.75$  seconds, so the rate is  $60/0.75 = 80$  bpm. This calculation assumes those detected landmarks correspond to consecutive relevant beats.

**Pause and try.** Why is the notebook's 48 ms output not automatically SDNN?

The code includes all detected RR intervals without selecting normal-to-normal intervals or removing detection artefacts. Its direct interpretation is the standard deviation of detected RR intervals.

**Pause and try.** What do the CNN's five false negatives mean?

Five of the 416 abnormal test examples were labelled normal. The abnormal recall is therefore  $411/416 \approx 98.8\%$ . The evaluation concerns the tutorial split from its source recording.

**Pause and try.** Why does an integer model still need a floating-point scale?

The scale defines the relationship between integer codes and represented values. It is metadata used to interpret tensors and configure computations. The model's storage and arithmetic scheme can still use integer tensors internally.

**Pause and try.** For the saved output scale and zero point, what does code 64 represent?

It represents  $(64 + 128)/256 = 0.75$ . A classifier threshold of 0.5 then selects abnormal for this output under the notebook's rule.

**Pause and try.** Why is a model that fits in flash not yet proven deployable?

It also needs supported operators, working RAM, application buffers, compatible preprocessing and acceptable runtime. These must be checked on the complete target application.

**Pause and try.** Why is the eight-move warehouse path shortest?

Reaching the opposite corner requires at least four row changes and four column changes. A valid eight-move path meets that lower bound. Its reward is 93 under the environment's undiscounted scoring rule.

**Pause and try.** Why does the goal square show zero value?

The episode ends on entry. The reward is received on the transition, and there is no subsequent continuation reward. The goal's action-value row is never trained as an ordinary decision state.

**Pause and try.** How do the taxi's 500 states arise?

There are 25 positions, five passenger situations and four destinations. Their product is 500. Six actions per state produce 3,000 Q-table entries.

**Pause and try.** Why does the neural name model have 11,897 parameters?

Its embedding has 270, its hidden dense layer 6,200, and its output layer 5,427. Their sum is 11,897. Flattening changes shape and adds no trainable parameters.

**Pause and try.** Does lower temperature make an answer true?

It concentrates probability on higher-scoring continuations. A high-scoring continuation can still be incorrect. Verify factual claims using appropriate evidence.

**Pause and try.** What should change for a stronger bigram-versus-neural comparison?

Split complete names before constructing examples. Fit both models using training names only and evaluate on the same held-out targets. Keep the final test names separate from tuning decisions.

## 26 Vocabulary reference

| Term            | Meaning in this lesson                                                |
|-----------------|-----------------------------------------------------------------------|
| Sampling rate   | Number of recorded samples per second.                                |
| ADC             | Circuit that maps an analogue voltage to a numerical code.            |
| Baseline wander | Slow drift in the recorded waveform's baseline.                       |
| Notch filter    | Filter with a narrow rejection band around a target frequency.        |
| RR interval     | Time between successive R-wave landmarks.                             |
| NN interval     | Interval between suitable consecutive normal beats.                   |
| Recall          | Fraction of relevant positive examples detected.                      |
| TinyML          | Machine-learning inference designed for constrained embedded systems. |
| Flash           | Nonvolatile storage for firmware and model bytes.                     |
| Tensor arena    | Preallocated working-memory region used by the micro-interpreter.     |
| Quantisation    | Approximate representation of values using a finite numerical grid.   |
| Scale           | Real-valued spacing associated with adjacent quantised codes.         |
| Zero point      | Integer code associated with real zero.                               |
| Calibration     | Estimating ranges from representative inputs for quantisation.        |
| Operator        | Runtime implementation of a computation such as convolution.          |
| Agent           | Decision-maker that selects actions.                                  |
| Environment     | System that responds to actions with observations and rewards.        |

| Term        | Meaning in this lesson                                                     |
|-------------|----------------------------------------------------------------------------|
| Policy      | Rule for selecting actions from available information.                     |
| Return      | Accumulated future reward, often discounted.                               |
| Q-value     | Estimate of return for a state-action choice.                              |
| Episode     | One interaction sequence from reset to an ending condition.                |
| Epsilon     | Probability of random exploration in an epsilon-greedy policy.             |
| Token       | Unit from a model's vocabulary, such as a character or word piece.         |
| Bigram      | Pair of adjacent tokens; the counting model conditions on one predecessor. |
| Embedding   | Learned vector representation associated with a token.                     |
| Logit       | Unnormalised score before probability normalisation.                       |
| Temperature | Positive scaling parameter applied to logits before sampling.              |
| RAG         | Retrieval of supporting material to condition generation.                  |
| RLHF        | Reinforcement learning using a signal derived from human feedback.         |

## 27 Coverage, provenance and references

### 27.1 Reading the source material accurately

These notes cover the supplied 40-slide deck, the speaker notes, all four notebooks and their exercises. Every nonempty notebook code cell appears with an explanation. Code typography has been normalised for the document, including ASCII equivalents for some display symbols. The Arduino listing is retained as a template and is discussed explicitly as incomplete deployment code.

Training outputs, converted model sizes and neural sample generations are attributed to the supplied files. Exercise results found only in the speaker notes are identified that way. The warehouse experiment and its two reward variants were repeated independently for these notes. Arithmetic for tensor shapes, parameter counts, quantisation and worked examples was checked separately.

The following distinctions are especially useful when teaching this material: the MIT-BIH recording and the prepared classifier dataset are separate inputs; the printed RR spread lacks normal-interval selection; host inference time is not board inference time; model bytes do not represent total RAM needs; and token-level validation is different from holding out complete names.

### 27.2 Primary references

Bracketed numbers in the explanations refer to the entries below. The user-supplied notebooks remain the source for their own code and saved outputs.

1. PhysioNet, MIT-BIH Arrhythmia Database: <https://physionet.org/content/mitdb/1.0.0/>.
2. TensorFlow, ECG tutorial data and autoencoder example: <https://www.tensorflow.org/tutorials/generative/autoencoder>.
3. Analog Devices, AD8232 product and evaluation guidance: <https://www.analog.com/en/products/ad8232.html>; <https://wiki.analog.com/resources/eval/ad8232-evaluation-guide/a03321a>.
4. SciPy, notch-filter design: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.iirnotch.html>.

5. SciPy, forward/backward filtering: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.filtfilt.html>.
6. SciPy, Butterworth filter design: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.butter.html>.
7. PhysioNet, normal-to-normal interval series: <https://physionet.org/content/chaos-heart-rate/1.0.0/>.
8. UEA/UCR time-series archive, ECG5000 source and preparation: <https://timeseriesclassification.com/description.php?Dataset=ECG5000>.
9. Espressif, ESP32 family specifications: [https://documentation.espressif.com/esp32\\_data\\_sheet\\_en.pdf](https://documentation.espressif.com/esp32_data_sheet_en.pdf).
10. Google AI Edge, integer quantisation workflow: [https://developers.google.com/edge/litert/conversion/tensorflow/quantization/post\\_training\\_integer\\_quant](https://developers.google.com/edge/litert/conversion/tensorflow/quantization/post_training_integer_quant).
11. Google AI Edge, quantisation specification: [https://developers.google.com/edge/litert/conversion/tensorflow/quantization/quantization\\_spec](https://developers.google.com/edge/litert/conversion/tensorflow/quantization/quantization_spec).
12. TensorFlow Lite Micro source and examples: <https://github.com/tensorflow/tflite-micro>.
13. Gymnasium, Taxi environment: [https://gymnasium.farama.org/environments/toy\\_text/taxi/](https://gymnasium.farama.org/environments/toy_text/taxi/).
14. Gymnasium, handling time limits: [https://gymnasium.farama.org/tutorials/gymnasium\\_basics/handling\\_time\\_limits/](https://gymnasium.farama.org/tutorials/gymnasium_basics/handling_time_limits/).
15. Mnih and colleagues, deep reinforcement learning: <https://arxiv.org/abs/1312.5602>.
16. Silver and colleagues, AlphaGo: <https://www.nature.com/articles/nature16961>.
17. Karpathy, makemore teaching project and name data: <https://github.com/karpathy/makemore>.
18. Bengio and colleagues, neural probabilistic language models: <https://jmlr.org/papers/v3/bengio03a.html>.
19. Ouyang and colleagues, instruction-following through human feedback: <https://arxiv.org/abs/2203.02155>.